



PDF Download
3695994.pdf
03 April 2026
Total Citations: 4
Total Downloads:
1291

Latest updates: <https://dl.acm.org/doi/10.1145/3695994>

RESEARCH-ARTICLE

Effective Hard Negative Mining for Contrastive Learning-Based Code Search

YE FAN, Nanjing University, Nanjing, Jiangsu, China

CHUANYI LI, Nanjing University, Nanjing, Jiangsu, China

JIDONG GE, Nanjing University, Nanjing, Jiangsu, China

LIGUO HUANG, Southern Methodist University, Dallas, TX, United States

BIN LUO, Nanjing University, Nanjing, Jiangsu, China

Open Access Support provided by:

Nanjing University

Southern Methodist University

Published: 23 February 2025

Online AM: 11 October 2024

Accepted: 12 August 2024

Revised: 10 August 2024

Received: 01 January 2024

[Citation in BibTeX format](#)

Effective Hard Negative Mining for Contrastive Learning-Based Code Search

YE FAN, CHUANYI LI, and JIDONG GE, National Key Laboratory for Novel Software Technology at Nanjing University, Nanjing, China
LIGUO HUANG, Department of Computer Science, Southern Methodist University, Dallas, TX, USA
BIN LUO, National Key Laboratory for Novel Software Technology at Nanjing University, Nanjing, China

Background. Code search aims to find the most relevant code snippet in a large codebase based on a given natural language query. An accurate code search engine can increase code reuse and improve programming efficiency. The focus of code search is how to represent the semantic similarity of code and query. With the development of code pre-trained models, the pattern of using numeric feature vectors (embeddings) to represent code semantics and using vector distance to represent semantic similarity has replaced traditional string matching methods. The quality of semantic representations is critical to the effectiveness of downstream tasks such as code search. Currently, the state-of-the-art (SOTA) learning method uses the contrastive learning paradigm. The objective of contrastive learning is to maximize the similarity between matching code and query (positive samples) and minimize the similarity between mismatched pairs (negative samples). To increase the reusing of negative samples, prior contrastive learning approaches use a large queue (memory bank) to store embeddings.

Problem. However, there is still a lot of room for improvement in using negative examples for code search: ① Due to the random selection of negative samples, semantic representations learned by existing models cannot distinguish similar codes well. ② Since semantic vectors in the memory bank are reused from previous inference results and then directly used for loss function calculation without gradient descent, the model cannot effectively learn the negative sample semantic information. *Method.* To solve the above problems, we propose a contrastive learning code search model with hard negative mining called CoCoHaNeRe: ❶ To enable the model to distinguish similar codes, we introduce hard negative examples into contrastive training, which are negative examples in the codebase that are most similar to positive examples. As a result, hard negative examples are most likely to make the model make mistakes. ❷ To improve the learning efficiency of negative samples during training, we add all hard negative examples to the model's gradient descent process. *Result.* To verify the effectiveness of CoCoHaNeRe, we conducted experiments on large code search datasets with six programming languages, as well as similar retrieval tasks code clone detection and code question answering. Experimental results show that our model achieves SOTA performance. In the code search task, the average MRR score of CoCoHaNeRe exceeds CodeBERT, GraphCodeBERT, and UniXcoder by 11.25%, 8.13%, and 7.38%, respectively.

This research/project was supported by the National Key Research and Development Program of China (2022YFF0711404), Natural Science Foundation of Jiangsu Province, China (BK20201250), CCF-Huawei Populus Grove Fund. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s).

Authors' Contact Information: Ye Fan, National Key Laboratory for Novel Software Technology at Nanjing University, Nanjing, China; e-mail: yefan@smail.nju.edu.cn; Chuanyi Li (corresponding author), National Key Laboratory for Novel Software Technology at Nanjing University, Nanjing, China; e-mail: lcy@nju.edu.cn; Jidong Ge (corresponding author), National Key Laboratory for Novel Software Technology at Nanjing University, Nanjing, China; e-mail: gjd@nju.edu.cn; LiGuo Huang, Department of Computer Science, Southern Methodist University, Dallas, TX, USA; e-mail: lghuang@smu.edu; Bin Luo, National Key Laboratory for Novel Software Technology at Nanjing University, Nanjing, China; e-mail: luobin@nju.edu.cn. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/2-ART76

<https://doi.org/10.1145/3695994>

It has also made great progress in code clone detection and code question answering. In addition, our method performs well in different programming languages and code pre-training models. Furthermore, qualitative analysis shows that our model effectively distinguishes high-order semantic differences between similar codes.

CCS Concepts: • **Software and its engineering** → **Search.based software engineering**; • **Information systems** → **Document representation**;

Additional Key Words and Phrases: Code Search, Contrastive Learning, Hard Negative Mining, Code Pre-trained Model

ACM Reference format:

Ye Fan, Chuanyi Li, Jidong Ge, LiGuo Huang, and Bin Luo. 2025. Effective Hard Negative Mining for Contrastive Learning-Based Code Search. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 76 (February 2025), 35 pages. <https://doi.org/10.1145/3695994>

1 Introduction

Code search involves the exploration of a vast code repository to identify relevant code snippets based on a **natural language (NL)** description of the desired functionality. Early research [7, 52] indicates that the efficiency of programming can experience significant enhancements through the reuse of code discovered via code search, resulting in substantial time and effort savings associated with repetitive coding tasks. Consequently, code search has emerged as a focal point of research and a key challenge within the realm of AI for Software Engineering.

To better align the semantics of NL and code, deep neural networks have been used to map code and its NL descriptions into similar sparse vectors [6, 10, 13, 24, 72, 87]. As shown in Figure 1, the similarity between these vectors is measured by cosine distances. In recent years, there has been notable success in the utilization of pre-trained code models, such as CodeBERT [21], GraphCodeBERT [26], and UniXCoder [25], in code representation learning, resulting in significant improvements in many code-related tasks, including code search. One of the current **state-of-the-art (SOTA)** models for code search is CoCoSoDa [73] (stands for Semantic Code Search with Multimodal Contrastive Learning and Soft Data Augmentation). Specifically, CoCoSoDa adopts multimodal contrastive learning [27] to minimize the distance between representations of positive (code, query) pairs and maximize the distance between representations of the query/code snippet and many other unpaired code snippets/queries. To learn a better sequence-level representation of the code snippets and queries, it also uses dynamic masking and replacing techniques [49] as a data augmentation approach, which is easy to build and scale to any **programming language (PL)**. The generated code snippets and queries are encoded by a momentum encoder [27], and the resulting embeddings are stored in a fixed-size memory bank. For each batch, the memory bank is updated with the previous batch and the transformation of the current batch.

However, relying solely on text similarity is insufficient to meet the requirements of code retrieval tasks. For instance, Figure 2 illustrates the retrieval results of the SOTA code retrieval method [73]. In this example, the user's query requires adding a callback function after the "create event" in the code. Nevertheless, the model returned incorrect results. The erroneous code (Hard Negative Example) exhibits higher text similarity with the user query, containing similar keywords such as "event," "callback," "create," and "version." In contrast, the correct code contains fewer similar keywords. Therefore, we observe that even SOTA code retrieval models lack the ability to distinguish between differences in code and text. To enable the model to discern semantic differences between similar codes, applying contrastive learning on samples with similar texts but different semantics is essential, rather than relying on simple random sampling.

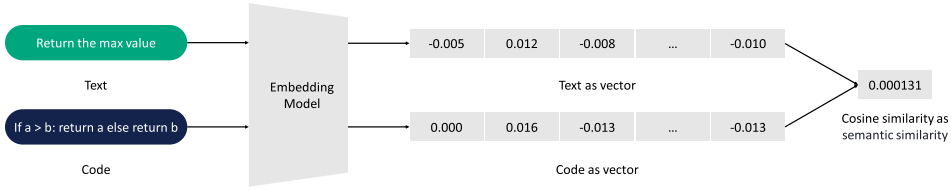


Fig. 1. A general framework for deep learning based code search.

Query: Adds a callback that records a version after a "create" event. Matching Code(Positive Example): <pre>def on_create @model_class.after_create { r r.paper_trail.record_create if r.paper_trail.save_version? } return if @model_class.paper_trail_options[:on].include?(:create) @model_class.paper_trail_options[:on] << :create end</pre>	Wrong Result(Hard Negative Example): <pre>def record_destroy(recording_order) return unless enabled? && !@record.new_record? in_after_callback = recording_order == "after" event = Events::Destroy.new(@record, in_after_callback) data = event.data.merge(data_for_destroy) version = @record.class.paper_trail.version_class.create(data) if version.errors.any? log_version_errors(version, :destroy) else assign_and_reset_version_association(version) end end</pre>
---	--

Fig. 2. Motivation example of hard negative mining in code search.

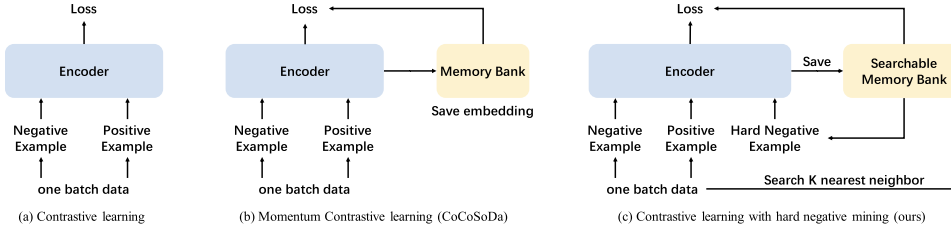


Fig. 3. Conceptual comparison between (a) Basic contrastive learning, (b) Momentum contrastive learning (CoCoSoDa), and (c) Contrastive learning with hard negative mining (the proposed CoCoHaNeRe).

Second, as shown in Figure 3, although the memory bank is continuously updated through the momentum encoder to alleviate its embedding inconsistency with the current batch in CoCoSoDa, the negative example from memory bank does not participate in gradient descent, which hinders the model from taking full advantage of the memory bank.

The contrastive learning method, which has achieved success in text and image matching tasks cannot be directly applied to code text matching tasks. This is because code and text essentially belong to the same modality, that is, strings, and only when code is converted into different forms such as **intermediate representation (IR)** or **abstract syntax tree (AST)** can it be regarded as two modalities. Because the two data are similar, it is more likely to produce hard negatives, so the model needs to learn high-order semantics to distinguish them. Relying solely on simple contrastive learning may restrict the model to learn only simple similarity matching.

However, simple text similarity alone cannot meet the requirements of code retrieval tasks. For example, Figure 2 shows the retrieval results of the SOTA code retrieval method [73]. In this example, the user's query requires adding a callback function after the "create event" in the code. However, the model returned incorrect results. The erroneous code (Hard Negative Example) has higher text similarity with the user query, containing similar keywords such as "event," "callback,"

“create,” and “version.” In contrast, the correct code contains fewer similar keywords. Therefore, we find that even SOTA code retrieval models still lack the ability to distinguish between code and text differences. To enable the model to discern semantic differences between similar codes, applying contrastive learning on samples with similar texts but different semantics is essential, rather than relying on simple random sampling.

In this paper, we propose a **Code search framework augmented by Contrastive learning with differentiable Hard Negative samples Retrieval (CoCoHaNeRe)**. We design a new memory bank that supports finding the nearest neighbor of query as hard negatives. Previous memory banks were simply queues to store embeddings, but to address the problem that contrastive learning can only learn low-level semantics, we need to filter out hard negatives that cannot be distinguished by simple semantic representation. We adopt differentiable hard negative embeddings for the model’s gradient descent process. Specifically, we recomputed the embeddings of hard negatives and put them in the same batch as positive examples for training. We find that previous methods that use embeddings to calculate the model loss function only use embeddings with no gradient information, which makes the learning of negatives slow and ineffective.

To evaluate the effectiveness of our method on the search task, we use the large-scale code search dataset **CodeSearchNet (CSN)** [34] and **XLCoST** [93] to conduct experiments. We compare our model with other PTM on CSN with six PLs. Experimental results show that our model outperforms other PTM on all language datasets after fine-tuning, and even outperforms some models’ fine-tune results in a zero-shot setting. Nonetheless, we also evaluate our method on other code understanding tasks involving code similarity. Experimental results show that our method achieves effective improvements on code question answering [31] and code clone detection [51, 74].

The main contributions of this work are as follows:

- We propose a new contrastive learning framework CoCoHaNeRe for code search and code similarity learning with hard negative mining. Our method contains a new searchable memory bank for hard negative retrieval and a new **hard negative contrastive loss (HNCL)** function. This allows the model to learn the semantic difference between similar code and text.
- We conduct experiments on multiple tasks and PTM. Our method achieves SOTA results on the code search task, and has significant improvement for different PLs and PTM. Case studies show that our method can indeed help the model learn high-order semantics.
- For better tool building and follow-up research, we open-source our code, experimental results, and data at [3].

The remainder of this paper is organized as follows. In Section 2, we present the background of code contrastive learning methods. In Section 3, we describe our approach in detail. In Section 4, we describe the settings of our experiments. In Section 5, we present the experimental results and analysis. In Section 6, we briefly review related works. In Section 7, we discuss the application scenario and limitations of our work. Finally, we conclude our work and present future work in Section 8.

2 Background

2.1 Traditional Code Search

Early code search approaches are mainly based on keywords matching, such as [28, 43, 50, 52, 60, 69]. To avoid noisy keywords, some works use query reformulation -based algorithms for code search. These techniques have applications in code search engines such as GitHub [2] and StackOverflow [4]. Some works utilized code IR [11, 14, 29, 42, 46, 55, 56, 63, 92]. For example, McMillan et al. [56] proposed Portfolio, which returns a chain of functions through keyword matching and PageRank

[70]. Lu et al. [50] extended a query with synonyms retrieved from WordNet and then conducted keyword matching of method signatures. Li et al. [46] introduced RACS, a code search framework for JavaScript that evaluates relationships among the invoked API functions such as sequencing, condition, and callback linkages.

However, these traditional approaches pay less attention to distinguish Code and NL from the modality perspective. Traditional keywords matching does not applicable to semantics of source code expressed through structure. Despite the matching algorithms [64] for codes use data structures such as graphs for pattern recognition, since they cannot be directly applied to NL, they cannot align the two modalities into a uniform representation space.

2.2 Deep Learning Based Code Search

Unlike traditional methods, deep learning-based approaches [6, 10, 13, 72, 87] successfully map code and text into high-dimensional feature vectors to represent their semantics. The distance between vectors can indirectly represent the similarity relationship between texts. These methods conduct supervised training directly on the datasets, which we call end-to-end methods. For example, NCS [72], proposed by Facebook [1], learns the embeddings of code using unsupervised learning. Zhu et al. [94] proposed OCoR, a code retriever using character level embedding and the overlap matrix to capture the overlaps between identifiers in code and words in NL descriptions.

Compared with these deep learning methods, the PTM contain more parameters due to the pre-training on large datasets, greatly improve the code search performance, and can even perform one-shot code search [25]. For example, PTM RoBERTa [49], CodeBERT [21], GraphCodeBERT [26], and UniXCoder [25] have been applied to code search and achieved the performance as much as 40% higher at most than that of traditional code search methods.

2.3 Code Search with Contrastive Learning

As shown in Figure 3(a), there are pre-training-based code search approaches utilizing contrastive learning as a pre-training task of the PTM. For example, SynCoBERT [79] utilizes the **multi-modal contrastive learning (MLC)** task where the positive and negative examples are pairs of either code and its documentation or code and its AST. CodeRetriever [45] also uses MLC but treats code-documentation pairs and code-comments pairs as positive/negative examples instead. Besides, ContraCode [36] proposed a reusable framework to augment training data for better leveraging contrastive learning in pre-training.

There are also code search methods utilizing contrastive learning during training (not pre-training) the model or fine-tuning a PTM. For instance, Corder et al. [79] proposed a contrastive learning method that simultaneously learns the similarity between code and text description, code and text summary, so as to simultaneously perform the model in two tasks: code search and code summary generation. CoCoSoDa [73] applies the MoCo [27] technology of the computer vision to the code search task. It uses a memory bank to save previous epoch embeddings, and employs a momentum encoder to ensure the continuity of embeddings in the memory bank. Although CoCoSoDa achieves SOTA of fine-tuned CodePTMs on code search, there are points where improvements could be made and they directly motivate us to propose the CoCoHaNeRe:

❶ CoCoSoDa treats all k embeddings (where k is much larger than the batch size) in memory banks as negative examples. Then, the examples distribution of positive and negative examples in each training step is imbalanced. Besides, among all the negative examples, only a small part of them are strong/hard negative examples (while the others are weak ones).¹ However, there is no need to learn

¹Strong/weak negative examples refer to examples whose items' similarities are very/slightly different from those of the items in positive examples.

from weak examples, since they are useless while we got hard examples. Moreover, the learning effect of hard negative examples may be reduced by the influence of the large number of weaker negative examples. With the training process of the model, the weak negative examples will be easily distinguished by the model, which makes the learning speed of the model gradually decrease in the later epochs. However, the hard negative examples are searched and will not decrease with the learning of the model, which is very beneficial for large-scale learning. In addition, the existing experimental datasets are relatively small. In large-scale datasets in real scenarios, there is a greater possibility of duplicate data. The more likely users will encounter similar code in the search process, so it is necessary to let the model learn the ability to distinguish these similar texts.

② *CoCoSoDa does not update the embeddings in the memory bank with the gradient descending during training.* Concretely, for one training step, CoCoSoDa first loads a batch of data, and the neural network first computes the loss function, then update the gradient of the network. Finally the model runs the gradient descent algorithm to update the parameters. After that, the gradients of all nodes will be cleared. Therefore, embeddings computed in previous training steps, as well as those saved in the memory banks, will become non-differentiable constants in subsequent training steps. As shown in Figure 3(c), the negative examples in memory bank were not incorporate into encoder's inference process but only in calculating the loss function. Since CoCoSoDa computes loss function directly using embeddings taken from memory bank, those embeddings cannot react to the gradients decent. Eventually, there will be representation gap between data in the batch and those in the memory bank.

3 Approach

In this section, we illustrate the proposed framework, CoCoHaNeRe, in detail. Generally, the current SOTA (shown in Figure 3(b)) inherits the basic contrastive approach (shown in Figure 3(a)) in handling the query and code in the current batch. Our CoCoHaNeRe (shown in Figures 3(c) and 4) also inherits the basic settings of CoCoSoDa, which can use a PTM as the base query/code encoder and first pre-train it on large dataset then fine-tune it on small dataset based on multimodal contrastive losses. However, differently, the query/code derived from the memory bank in CoCoHaNeRe goes through a trainable encoder to participate in gradient descent. Specifically, CoCoHaNeRe consists of the following four parts:

- *Searchable and Updateable Memory Bank* is a fixed-size First-In-First-Out queue for storing embeddings of queries/codes that will be used in constructing negative examples for the current batch. We use a Query Memory Bank (for retrieving similar queries for a given code) and a Code Memory Bank (for retrieving similar codes for a given query). In addition to the basic *push* operation (i.e., inserting new items from the tail of the queue and removing excess items from the head if the queue is full), our memory bank supports (1) the *search* operation: deriving k nearest neighbors (i.e., knn) from the bank for a given embedding, and (2) the *update* operation: replacing the embeddings at the specified positions with given new ones.
- *Shared Query/Code Encoder* is used to map the query/code text to a high-dimensional vector representation (i.e., embedding). As emphasized above, in our model, not only the query/code of the current batch will be calculated by the encoder, but the top k similar queries/codes retrieved from the memory bank will also be recalculated by the encoder based on their original texts. Therefore, all positive and negative examples will participate in the model's gradient calculation. We adopt encoders of code PTM, such as CodeBERT, GraphCodeBERT, and UniXCoder, as the shared encoder.
- **Bimodal contrastive loss (BCL)** is calculated purely based on the current batch. The positive examples are the initial (Query, Code) pairs in the current batch, and the negative examples

are obtained by combining the Query in each positive example with the Code from all other positive examples.

- *HNCL* is calculated by combining the Query/Code retrieved from the memory bank and those in the current batch. Positive examples are the initial (Query, Code) pairs in the current batch. Hard negative examples are obtained by combining each Query/Code in the positive examples with its top- k similar Code/Query derived from the Code/Query memory bank.

3.1 Searchable and Updateable Memory Bank

Essentially, the basic data structure of the memory bank is a list and each data instance in the Query/Code memory bank (recall that we have memory banks for queries and codes separately) is a (DataID, QueryEmbedding/CodeEmbedding) pair. The DataID corresponds to the DataID in the training set where each data instance is a triplet (DataID, Query Text, Code Text). Since the *push*, *search* and *update* operations on the memory bank are frequently conducted during training, its access efficiency directly affects the overall training efficiency. So, to improve its operation efficiency, we use GPU memory for memory banks (by explicitly specifying to store the customized Memory Bank on a concrete GPU device in PyTorch). Naturally, the *push*, *search* and *update* operations are also accomplished by the GPU. Concretely, to *push* N (DataID, Embedding) pairs in the bank, we just append them to the list and remove the first N pairs from the bank. To *search* k nearest neighbors for a QueryEmbedding/CodeEmbedding, we first calculate its cosine similarity with all embedding in the bank, then utilize torch. Top- k provided by PyTorch to retrieve top- k similarities, and eventually return both Indexes (i.e., position in the memory bank) and DataIDs of the k most similar embeddings. To *update* with (DataID, Embedding) pairs, Indexes of the embeddings to be updated should also be given.

3.2 Shared Query/Code Encoder

We follow CoCoSoDa by inserting a special token [CLS] at the beginning of the input code/query sequence, using the embedding of [CLS] at the last layer as the whole sequence-level representation. Additionally, we adopt a 2-layer Multi-Layer Perception projector to map the sequence-level representation of code/query to a shared latent space.

For each current batch, the query/code will first go through the encoder to generate an embedding. Then, these embeddings will be used to retrieve k nearest neighbors from memory banks using the *search* operation. As mentioned, only Indexes and DataIDs will be returned by *search*, meaning that embeddings in the memory bank are not used in calculating losses for the current batch. Instead, DataIDs are used to retrieve the initial text of the most similar queries/codes from the training set, and they are put into the encoder to generate new embeddings for them. These embeddings and those of the current batch are then used in the gradient descent of the model. This is a key difference between CoCoHaNeRe and existing memory bank-based code search approaches, where the embeddings of code/query in the memory bank are either fixed and stored in advance or slowly updated with a momentum encoder.

There are two considerations for recalculating the embeddings in the memory bank. On the one hand, the number of queries/codes obtained from the memory bank (k) is much less than that of existing approaches. It has been proven that a small number of embeddings in the memory bank would have little effect on gradient descent during backpropagation if they were used in calculating contrastive losses directly, without going through the trainable encoder [27]. On the other hand, the new embeddings generated by the encoder can be written back to the memory bank using the *update* operation (as shown in Figure 4), to continuously update the content in the memory bank. Keeping the embeddings in the memory bank up-to-date is beneficial for ensuring the effectiveness

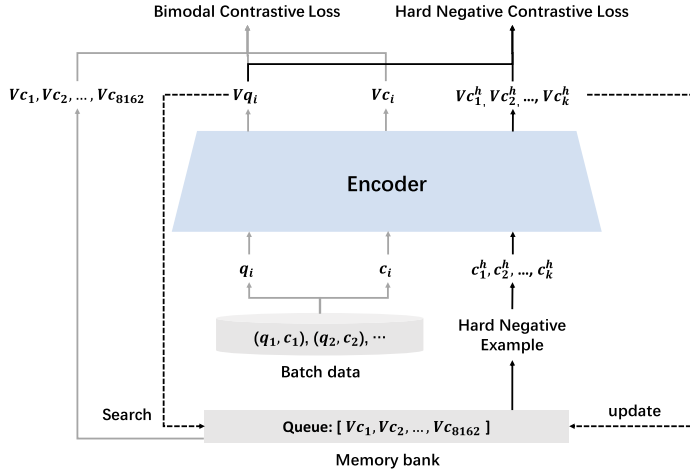


Fig. 4. Framework of CoCoHaNeRe. For clarity, we only show the Code Memory Bank and the process of retrieving hard negatives with queries. In practice, we also have a Query Memory Bank and the process of retrieving hard negatives with codes. The implementation of the two parts are completely symmetrical.

of the retrieved k nearest neighbors. Additionally, the embeddings of the current batch will also be written to the memory bank using the *push* operation.

3.3 BCL

We split the contrastive loss into two parts. The first one is calculated with queries and codes from the current batch and we name it BCL. Let $[(q_1, c_1), \dots, (q_n, c_n)]$ be positive (Query, Code) pairs of a batch, then with similarity measured by dot product [73], the BCLs for the query q_i and code c_i ($1 \leq i \leq n$) are defined separately as:

$$L_{q_i}^B = -\log \frac{\exp(v_{q_i} \cdot v_{c_i})}{\sum_{j=1}^n \exp(v_{q_i} \cdot v_{c_j})}, \quad (1)$$

$$L_{c_i}^B = -\log \frac{\exp(v_{c_i} \cdot v_{q_i})}{\sum_{j=1}^n \exp(v_{c_i} \cdot v_{q_j})}, \quad (2)$$

where v_q and v_c denote the embedding of the query q and code c given by the encoder. The optimization objective is to maximize the semantic similarity of the query and its paired code snippet and minimize the semantic similarity of the query and its unpaired code snippets. Figure 5(a) intuitively shows the positive and negative examples for calculating the BCL.

We calculate the overall bimodal contrastive learning loss for the batch as:

$$L^B = \sum_{i=1}^n \frac{L_{q_i}^B + L_{c_i}^B}{2}. \quad (3)$$

3.4 HNCL

The second part of the contrastive loss is calculated by combining the current batch and the memory bank and we name it HNCL. Let (q_i, c_i) be a positive pair in the batch, $[c_1, c_2, \dots, c_k]$ and $[q_1, q_2, \dots, q_k]$ are initial texts (not embedding)² of top- k similar codes for q_i in the Code Memory

²Recall that we use DataIDs stored in the memory bank to retrieve initial text of query and code and re-encoding them, instead of using embedding in the memory bank directly.

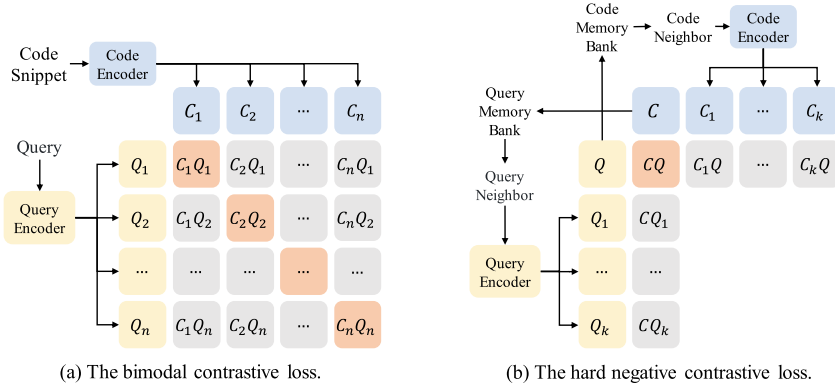


Fig. 5. Positive and Negative examples of contrastive losses.

Bank and top- k similar queries for c_i in the Query Memory Bank, respectively (We also try searching similar queries for q_i and similar codes for c_i to construct negative examples, but the performance is not as high as the adopted method. Concrete experimental results of different searching strategies are shown in Section 5.3.). Then, following the BCL, we define the HNCL for the query q_i and code c_i as:

$$L_{q_i}^{HN} = -\log \frac{\exp(v_{c_i} \cdot v_{q_i})}{\exp(v_{c_i} \cdot v_{q_i}) + \sum_{j=1}^k \exp(v_{q_i} \cdot v_{c_j})}, \quad (4)$$

$$L_{c_i}^{HN} = -\log \frac{\exp(v_{c_i} \cdot v_{q_i})}{\exp(v_{c_i} \cdot v_{q_i}) + \sum_{j=1}^k \exp(v_{c_i} \cdot v_{q_j})}. \quad (5)$$

The overall HNCL for the batch is defined as:

$$L^{HN} = \sum_{i=1}^n \frac{L_{q_i}^{HN} + L_{c_i}^{HN}}{2} \quad (6)$$

Notice that v_{c_j} and v_{q_j} are embedding of c_j and q_j re-encoded by the encoder. If we reuse the the embedding in the memory banks as v_{c_j} and v_{q_j} , they are constants in the loss functions. Then, the $\sum_{j=1}^k \exp(v_{q_i} \cdot v_{c_j})$ and $\sum_{j=1}^k \exp(v_{c_i} \cdot v_{q_j})$ in $L_{q_i}^{HN}$ and $L_{c_i}^{HN}$ are less trainable, which means that the loss L^{HN} would be optimized slower and, in contrary, it would contribute less to the gradient descent of the model. Besides, considering that the k may be much smaller than the batch size, the impact of L^{HN} on the model would even smaller than that of L^B . For these reasons, in CoCoHaNeRe, we let hard negative examples fully participating in the gradient descent to accelerate the training procedure.

Eventually, for the total loss of the model, we just use the sum of the bimodal and HNCLs:

$$L = L^B + L^{HN}. \quad (7)$$

3.5 Pseudocode of Training Procedure

Putting all the aforementioned parts together, the complete training procedure of CoCoHaNeRe is clear and is shown in Algorithm 1 in the form of pseudocode. The method of searching hard negatives in the memory bank is shown in Algorithm 2.

According to the pseudocode in Algorithm 1, one more thing deserves mention. When retrieving similar queries/codes for the current batch from the memory bank, the inputs are embeddings

Algorithm 1: Pseudocode of CoCoHaNeRe Pre-Training Process

Input: Backbone encoder E , memory bank M and code search dataset.

Output: the pre-trained code search model.

- 1: **for** Load a batch of code and query pairs from dataset **do**
 - 2: Obtain embeddings of queries v_q and codes v_c respectively in the batch using E .
 - 3: Calculate the bimodal contrastive loss L^B using v_q and v_c through formula Equation (3).
 - 4: Find K-Near-Neighbors of v_q and v_c in memroy bank m as hard negatives examples through *search* operation in Algorithm 2.
 - 5: Load the inputs of hard negatives from dataset and calculate their embeddings using E .
 - 6: Calculate the hard negative contrastive loss L^{HN} through formula Equation (6).
 - 7: Calculate the total loss L through formula Equation (7).
 - 8: Do back propagation and update the parameters of optimizer.
 - 9: Update memory bank M with the embeddings of the current batch using *push* operation.
 - 10: Update memory bank M with the embeddings of hard negatives using *update* operation.
 - 11: **end for**
-

Algorithm 2: Pseudocode of Find Hard Negatives in Memroy Bank

Input: Code or query embeddings v and memory bank M .

Output: the k-nearest-neighbors' embedding and index of dataset.

- 1: Concatenate all embeddings in M .
 - 2: Calculate the cosine similarity between v and all embeddings in M .
 - 3: Sort the cosine similarity in descending order.
 - 4: Return the top-k embeddings and their indexes in M .
-

encoded with the latest encoder, but the embeddings in the memory bank are encoded with older encoders. The inconsistency of the two types of embeddings results in the retrieval of k similar queries/codes that are not genuinely the most similar ones. To obtain the real similar ones, all embeddings in the memory bank should first be updated with the latest encoder. But this is computationally expensive and will slow down the training speed. In CoCoHaNeRe, this inconsistency problem is alleviated to a certain extent by dynamically updating some embeddings in the memory bank during each batch. Perhaps some specific strategies can be designed (for example, after a certain number of batches, update the entire memory bank with the latest encoder) to further alleviate this inconsistency, but our current setting does not prevent CoCoHaNeRe from achieving a new SOTA for the code search task. We leave this as future work.

4 Experimental Setup

Here, we first describe our experimental dataset and present the baselines used in our experiments. Then, we introduce the evaluation metrics and other implementation details.

4.1 Datasets

Code Search. To evaluate the performance of CoCoHaNeRe, we conduct experiments on CSN [34]. CSN contains six subdatasets corresponding to six different PLs (i.e., Ruby, Java, Python,

Table 1. Dataset Statistics

Language	Training	Validation	Test	Candidate Code
Ruby	24,927	1,400	1,261	4,360
JavaScript	58,025	3,885	3,291	13,981
Java	164,923	5,183	10,955	40,347
Golang	167,288	7,325	8,122	28,120
PHP	241,241	12,982	14,014	52,660
Python	251,820	13,914	14,918	43,827

JavaScript, Golang, PHP), and each data instance is a pair of a code snippet and its corresponding text description. CSN has two versions: one is undocumented code data (for pre-training), and another is filtered documented code data (for fine-tuning). The statistics of the dataset are shown in Table 1.

To evaluate the performance of our model in real-world scenarios, we conducted experiments on the XLCoS dataset [93], which contains real-world code search examples. XLCoS, collected from GeeksForGeeks, comprises programming questions along with their corresponding answers and code. It includes 8 languages (7 PLs and English) and covers 10 cross-lingual code tasks. XLCoS is currently the largest parallel dataset for source code, both in terms of size and the number of languages. We utilized the NL-to-code dataset within the code retrieval subset of XLCoS to evaluate our model.

Code Question Answering. Code question answering involves returning the code that can answer a user's question. This task relies on learning the similarity between code and text. We evaluated the effectiveness of our method on question answering using CoSQA dataset [31]. This dataset comprises 20,604 labels for pairs of NL queries and codes, each annotated by at least 3 human annotators. Each label indicates whether the code can answer the query or not. The queries are derived from the search logs of the Microsoft Bing search engine, and the code is a function from GitHub. It's important to note that code question answering can only return the unique correct result, rather than the similarity between the returned codes.

Clone Detection. Clone detection includes two subtasks. The first subtask is to predict whether two given codes have the same semantics. We use the BigCloneBench dataset [74] for this subtask. The second subtask aims to retrieve semantically similar codes given a code as the query. We use the POJ-104 dataset [58] to perform this task.

BigCloneBench is a large code clone benchmark that contains over 6,000,000 true clone pairs and 260,000 false clone pairs from 10 different functionalities. The dataset provided by Wang et al. [78] is filtered by discarding code fragments without any tagged true or false clone pairs, leaving it with 9,134 Java code fragments. Finally, the dataset includes 901,028/415,416/415,416 examples for training, validation and testing, respectively.

POJ-104 dataset [58] comes from a pedagogical programming **open judge (OJ)** system that automatically judges the validity of submitted source code for specific problems by running the code. It consists of 104 problems and includes 500 student-written C/C++ programs for each problem. Unlike the BigCloneBench dataset, the task of POJ104 aims to retrieve other programs that solve the same problem given a program.

4.2 Baselines

The following code search approaches are treated as the baseline for evaluating CoCoHaNeRe. All the results are compared to the results obtained by running their open source code or model parameters in our environment setting. For methods with open source code, we use their open source code to run experiments in our experimental environment. For models without open source code, if they publish their model parameters, we use their model to run in our test code and experimental environment. For works that do not open their model parameters, we reproduce their methods and conduct experiments in our experimental environment.

- IR-based methods include *BOW* [53], *TF-IDF* [71], and *Jaccard* [35]: BOW and TF-IDF utilize bag-of-word and frequency-inverse document frequency algorithms. They measure similarities by cosine similarities. Jaccard searches the code snippet for a query according to the Jaccard similarity coefficient.
- RoBERTa* and *RoBERTa-Code*: They are transformer encoder-based pre-training models that are pre-trained on a large text dataset and a code dataset (CSN), respectively.
- CodeBERT*, *GraphCodeBERT*: CodeBERT is a PTM of a bi-directional RoBERTa-based coding language, using **masked language modeling (MLM)** and replaced Token detection task for pre-training. GraphCodeBERT codes the structure information AST of the code and uses MLM, dataflow edge prediction and node alignment training tasks for pre-training.
- CodeT5* [82], *CodeT5+* [80]: CodeT5 is pre-trained with three identifier-aware pre-training tasks to enable the model to identify identifiers in source code or recover masked identifiers. CodeT5+ is an upgraded version of CodeT5. It incorporates more pretraining tasks such as span denoising, causal language modeling and text-code matching for code representation. It includes multiple versions with different parameter sizes. For fair comparison, we choose to compare CodeT5p-220m with other models.
- PLBART* [5], *SPT-Code (SPT)* [62]: The PLBART model is pre-trained with denoising autoencoding, which is used to reconstruct the corrupted input code sequence. The SPT-Code model takes source code, corresponding AST and paired summarization as input and is pre-trained with three code-specific tasks.
- SyncoBERT* [79] and *UniXcoder* [25]: SyncoBERT takes code snippet, AST and code summary as input and was pre-trained with identifier prediction and AST edge prediction to learn syntactic knowledge. UniXcoder is a unified cross-modal PTM that leverages multi-modal contents, i.e. code comment and AST. UniXcoder provides significant improvement on zero-shot code search.
- CoCoSoDa* [73]: A work that uses multimodal contrastive learning to improve code search tasks. Compared to our algorithm, only the basic memory bank and the traditional MoCo momentum encoder are used.

We did not adopt OpenAI's embedding model [59] as a baseline. This is because OpenAI's model was trained on all GitHub data, indicating that the test set data might have leaked into the training set, thus preventing us from ensuring a fair comparison. For the CodeT5+ model, the original authors utilized a single layer of attention for reordering. To fairly compare the performance, we only tested the encoder part of the CodeT5+ model. Both our model and the baseline models underwent the same training process, beginning with pre-training on the entire dataset, followed by fine-tuning on individual language datasets. During the pre-training phase, our model employed the method described in this paper, while all other settings remained identical to the baselines. The number of training epochs may vary as we employed an early stopping mechanism to prevent overfitting, and halting training when the validation performance began to decline.

4.3 Evaluation Metrics

We use two evaluation metrics for code search task, **mean reciprocal rank (MRR)** and **Top-K Precision (P@K, $K = 1, 5, 10$)**, which are widely used in all code search tasks.

Top-K Precision is the rank of the paired code snippet related to the current query. I is the indicator function that returns 1 if $Rank \leq K$ otherwise returns 0. Given the queries amount as D . It's defined as follows:

$$R@K = \frac{1}{|D|} \sum_{i=1}^{|D|} I(Rank_i \leq K). \quad (8)$$

MRR is the average of the reciprocal of ranks. It's calculated as follows:

$$MRR = \frac{1}{|D|} \sum_{i=1}^{|D|} \frac{1}{Rank_i}. \quad (9)$$

Normalized discounted cumulative gain (NDCG) [81] is a metric commonly used for retrieval and recommendation tasks. However, in the context of code retrieval, which is a one-to-one task, NDCG and MRR have similar performance as the gain in NDCG is only calculated for the correct match. Therefore, it is unnecessary to use both metrics.

For the POJ-104 dataset in the code clone detection task, we use **Mean Average Precision (MAP)** for evaluation. MAP is a variant of mean precision, which is the average of the mean precision for all queries. The calculation formula of MAP is as follows:

$$MAP = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{|R_i|} \sum_{j=1}^{|R_i|} \frac{P(j)}{j}, \quad (10)$$

where Q is the set of queries, R_i is the set of retrieved results for query i , and $P(j)$ is the precision of the top j retrieved results for the i th query.

For the BigCloneBench dataset, we use *Precision* and *Recall* as evaluation metrics. Precision is the ratio of the number of true positive clone pairs to the number of all clone pairs. Recall is the ratio of the number of true positive clone pairs to the number of all true clone pairs. We also use *F1* score as the evaluation metric for code clone detection. F1 score is the harmonic mean of precision and recall, which is defined as follows:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}. \quad (11)$$

For the CoSQA dataset, we use accuracy as the evaluation metric. *Accuracy* is the ratio of the number of correct answers to the number of all answers.

4.4 Other Settings

We choose CodeT5+ as our backbone encoder for training. In order to ensure the fairness of the experiments, we strictly follow the settings of the original papers of each PTM, such as we use a learning rate of $3e^{-5}$ and AdamW as the optimizer. The vocabulary sizes of code and queries are set to 51,451. Max sequence lengths of code snippets and queries are 128 and 256, respectively. The training step of multimodal contrastive learning stage is 100K and the maximum epochs of fine-tune stage is 5. Our memory bank size is set to 8,096 and we set neighbor amount k as 10. Since the number of training epochs for different methods is different, in order to ensure fairness, we set training epoch as 10 which is the minimum number of training epochs. Our batch size is set to 32. All experiments were run on 4*NVIDIA Tesla V100 32GB GPUs.

To prevent overfitting of the model, we employ the following methods. Firstly, we utilize three data augmentation techniques: ① We apply random masking to tokens in both the code and query. ② We employ random slicing to extract a portion of the long text. ③ We use random replacement to replace tokens in some parts of the text. Secondly, we introduce noise into the embedding layer of the model. Finally, we employ the early stopping technique, while training, we concurrently test the model on the validation set. When the performance on the validation set begins to decline, we halt the model training.

5 Experiments

We have designed several experiments to answer the following research questions:

- RQ1: What is the effectiveness of CoCoHaNeRe?*
- RQ2: What is the performance of our approach on other PTM?*
- RQ3: How much do different components contribute?*
- RQ4: What is the impact of different hyperparameters?*
- RQ5: How does CoCoHaNeRe perform on other code retrieval tasks?*
- RQ6: Can CoCoHaNeRe be applied to real world scenario?*
- RQ7: How does CoCoHaNeRe perform on low-resource scenario?*

5.1 Effectiveness of CoCoHaNeRe Compared with Baseline Methods (RQ1)

Quick View of Answers and Implications

CoCoHaNeRe outperforms other baseline code search methods and achieves new SOTAs on all the adopted datasets. This indicates that our method can better learn the matching between code and text, and can be applied to code search tasks in different languages. PTM with encoder architecture for code and text matching perform best. As a result, CoCoHaNeRe is more suitable for using encoder architecture PTM with code and text pre-training tasks.

Method: To evaluate the effectiveness of our method, we train our CoCoHaNeRe on CSN according to the settings mentioned in Section 4. We first pre-train our CoCoHaNeRe on the dataset containing all languages, and then fine-tune it on the dataset of a single language. We use MRR as the evaluation metric. For special models such as GraphCodeBERT, the input requires code structure information, so we filter out the code that cannot generate **control flow graph (CFG)** in the dataset. For each code and text pair, we remove the comment information and truncate them according to the length mentioned in the setup.

Results: The results are shown in Table 2. The best experimental results are shown in bold. CoCoHaNeRe performs better than all baseline approaches. Compared to IR-based methods, our method improves performance by more than 40%. Compared to PTM without code search task, our method improves performance by more than 4%. This shows that our method achieves good results on the code retrieval task.

The results show that using deep learning methods exceeds traditional IR methods by about 10%. Models pre-trained on code exceed NL pre-training models by about 10%. Compared to simple PTM Roberta, Roberta(code) model, our method improves the effect by more than 10%. Compared with other more complex code pre-training models, our method exceeds about 6%. Compared with CoCoSoDa, our method exceeds about 4%.

We found that the test effect of the Go language dataset is higher, reaching more than 90%. Although the JavaScript dataset is lower, generally less than 20% of the results of Go. But at the

Table 2. MRR Values of Different Code Search Approaches

Model	Java	Python	PHP	JS	Ruby	Go	Avg
Jaccard	0.235	0.243	0.182	0.191	0.220	0.345	0.236
Bow	0.245	0.222	0.193	0.184	0.230	0.350	0.237
TF-IDF	0.262	0.240	0.215	0.204	0.239	0.363	0.254
RoBERTa	0.605	0.590	0.561	0.523	0.587	0.855	0.637
RoBERTa-Code	0.636	0.621	0.581	0.570	0.631	0.864	0.651
CodeBERT	0.677	0.672	0.626	0.621	0.679	0.885	0.693
GraphCodeBERT	0.691	0.692	0.649	0.644	0.703	0.897	0.713
CodeT5	0.686	0.698	0.645	0.655	0.719	0.888	0.715
CodeT5+	0.703	0.710	0.660	0.658	0.722	0.908	0.729
PLBART	0.663	0.663	0.611	0.616	0.675	0.887	0.685
SPT-Code	0.700	0.699	0.651	0.641	0.701	0.895	0.715
SynCoBERT	0.715	0.717	0.670	0.677	0.722	0.913	0.735
UniXcoder	0.688	0.715	0.650	0.644	0.708	0.903	0.718
CoCoSoDa	0.718	0.719	0.671	0.676	0.736	0.914	0.739
CoCoHaNeRe	0.764 ↑6.41%	0.756 ↑5.15%	0.711 ↑5.96%	0.714 ↑5.62%	0.755 ↑2.58%	0.927 ↑1.42%	0.771 ↑4.35%

Statistical significance of experiments: $p < 0.01$.

same time, on the Go language dataset, our method has the smallest improvement, about 1%. On the Java dataset, our method has the largest improvement, about 6%.

Discussion: Our method outperforms all other PTM. The experimental results show that our method can better learn the matching between code and text compared to other PTM. At the same time, significant improvements have been made on the datasets of all languages compared to the original backbone effect. We found that the effect of code pre-training models far exceeds other traditional methods. Due to the large number of parameters and pre-training tasks of the model, the semantic information learned far exceeds the effect of text matching. The model using the encoder architecture performs better than the model using the encoder-decoder architecture. This shows that using the decoder is indeed not conducive to the learning of code understanding tasks.

In Figure 6, we randomly selected 80 examples (query, code) from the CSN dataset, which were correctly matched by CoCoHaNeRe and incorrectly matched by CoCoSoDa, and we saved their embeddings generated by each model. The T-SNE visualization shows that the embeddings generated by CoCoHaNeRe, are more one-to-one, while the embeddings of the matched code and query pairs generated by CoCoSoDa are often further apart. This shows that CoCoSoDa, which does not consider hard negatives, cannot distinguish them well.

5.2 Effectiveness of Other Backbone Encoder (RQ2)

Quick View of Answers and Implications

Our approach is orthogonal to the pre-trained techniques for improving performance in code search tasks and can significantly enhance the performance of existing PTM. Our method shows the most substantial improvement in the top-1 position. Additionally, its impact on CodeBERT even surpasses that of other models. We recommend employing our method in scenarios with more redundant data, as it can more accurately retrieve the correct code in such cases.

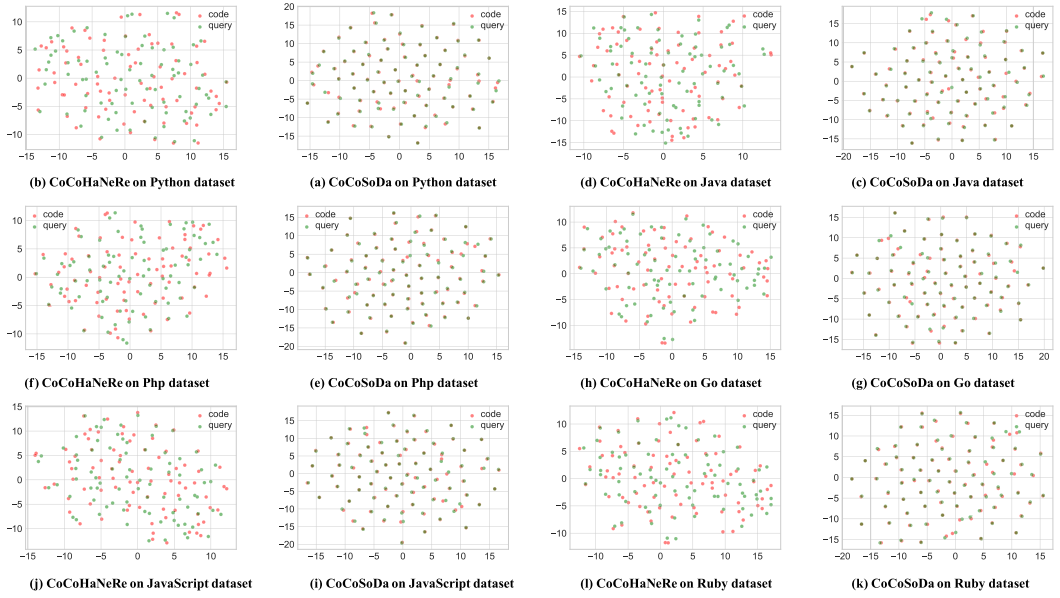


Fig. 6. Visualization of tsne representations of embedding of hard negative samples selected from CSN generated by CoCoSoDa and CoCoHaNeRe.

Table 3. Top- k Precision of Different Code Search Approaches

PL	Metric	ROB	ROB+CCHNR	COB	COB+CCHNR	GCB	GCB+CCHNR	UNI	CCSD	UNI+CCHNR
Ruby	MRR	0.587	0.714 ($\uparrow 21.64\%$)	0.679	0.749 ($\uparrow 10.31\%$)	0.703	0.721 ($\uparrow 2.56\%$)	0.688	0.718	0.741 ($\uparrow 7.70\%$)
	R@1	0.469	0.624 ($\uparrow 33.05\%$)	0.583	0.656 ($\uparrow 12.52\%$)	0.607	0.632 ($\uparrow 4.12\%$)	0.576	0.622	0.646 ($\uparrow 12.15\%$)
	R@5	0.717	0.827 ($\uparrow 15.34\%$)	0.800	0.865 ($\uparrow 8.13\%$)	0.824	0.828 ($\uparrow 0.49\%$)	0.818	0.841	0.864 ($\uparrow 5.62\%$)
	R@10	0.785	0.874 ($\uparrow 11.34\%$)	0.853	0.900 ($\uparrow 5.51\%$)	0.872	0.873 ($\uparrow 0.11\%$)	0.875	0.891	0.905 ($\uparrow 3.42\%$)
JS	MRR	0.523	0.636 ($\uparrow 21.60\%$)	0.621	0.679 ($\uparrow 9.34\%$)	0.644	0.661 ($\uparrow 2.64\%$)	0.644	0.676	0.700 ($\uparrow 8.70\%$)
	R@1	0.413	0.540 ($\uparrow 30.75\%$)	0.514	0.576 ($\uparrow 12.06\%$)	0.538	0.566 ($\uparrow 5.20\%$)	0.537	0.572	0.606 ($\uparrow 12.85\%$)
	R@5	0.652	0.754 ($\uparrow 15.64\%$)	0.752	0.802 ($\uparrow 6.65\%$)	0.774	0.780 ($\uparrow 0.78\%$)	0.774	0.794	0.817 ($\uparrow 5.56\%$)
	R@10	0.730	0.812 ($\uparrow 11.23\%$)	0.814	0.849 ($\uparrow 4.30\%$)	0.834	0.835 ($\uparrow 0.12\%$)	0.834	0.851	0.868 ($\uparrow 4.08\%$)
Go	MRR	0.855	0.904 ($\uparrow 5.73\%$)	0.885	0.913 ($\uparrow 3.16\%$)	0.897	0.904 ($\uparrow 0.78\%$)	0.903	0.914	0.922 ($\uparrow 2.10\%$)
	R@1	0.800	0.859 ($\uparrow 7.38\%$)	0.837	0.870 ($\uparrow 3.94\%$)	0.858	0.862 ($\uparrow 0.47\%$)	0.854	0.871	0.885 ($\uparrow 3.63\%$)
	R@5	0.926	0.960 ($\uparrow 3.67\%$)	0.944	0.965 ($\uparrow 2.22\%$)	0.954	0.956 ($\uparrow 0.21\%$)	0.963	0.965	0.968 ($\uparrow 0.52\%$)
	R@10	0.949	0.973 ($\uparrow 2.53\%$)	0.962	0.978 ($\uparrow 1.66\%$)	0.972	0.972 ($\uparrow 0\%$)	0.976	0.978	0.979 ($\uparrow 0.31\%$)
Python	MRR	0.590	0.684 ($\uparrow 15.93\%$)	0.672	0.720 ($\uparrow 7.14\%$)	0.692	0.703 ($\uparrow 1.59\%$)	0.715	0.719	0.747 ($\uparrow 4.48\%$)
	R@1	0.480	0.585 ($\uparrow 21.88\%$)	0.574	0.622 ($\uparrow 8.36\%$)	0.594	0.601 ($\uparrow 1.18\%$)	0.616	0.617	0.653 ($\uparrow 6.01\%$)
	R@5	0.727	0.806 ($\uparrow 10.87\%$)	0.792	0.839 ($\uparrow 5.93\%$)	0.813	0.831 ($\uparrow 2.21\%$)	0.836	0.841	0.861 ($\uparrow 2.99\%$)
	R@10	0.793	0.859 ($\uparrow 8.32\%$)	0.850	0.885 ($\uparrow 4.12\%$)	0.866	0.882 ($\uparrow 1.85\%$)	0.889	0.893	0.903 ($\uparrow 1.57\%$)
Java	MRR	0.605	0.702 ($\uparrow 16.03\%$)	0.677	0.732 ($\uparrow 8.12\%$)	0.691	0.714 ($\uparrow 3.33\%$)	0.688	0.718	0.749 ($\uparrow 8.87\%$)
	R@1	0.499	0.607 ($\uparrow 21.64\%$)	0.580	0.642 ($\uparrow 10.69\%$)	0.592	0.622 ($\uparrow 5.07\%$)	0.582	0.624	0.663 ($\uparrow 13.92\%$)
	R@5	0.737	0.819 ($\uparrow 11.13\%$)	0.796	0.843 ($\uparrow 5.90\%$)	0.817	0.828 ($\uparrow 1.35\%$)	0.821	0.837	0.854 ($\uparrow 4.02\%$)
	R@10	0.796	0.866 ($\uparrow 8.79\%$)	0.852	0.890 ($\uparrow 4.46\%$)	0.865	0.872 ($\uparrow 0.81\%$)	0.875	0.884	0.897 ($\uparrow 2.51\%$)
Php	MRR	0.561	0.648 ($\uparrow 15.51\%$)	0.626	0.675 ($\uparrow 7.83\%$)	0.649	0.668 ($\uparrow 2.93\%$)	0.65	0.671	0.701 ($\uparrow 7.85\%$)
	R@1	0.450	0.546 ($\uparrow 21.33\%$)	0.520	0.573 ($\uparrow 10.19\%$)	0.545	0.569 ($\uparrow 4.40\%$)	0.541	0.567	0.603 ($\uparrow 11.46\%$)
	R@5	0.694	0.773 ($\uparrow 11.38\%$)	0.753	0.800 ($\uparrow 6.24\%$)	0.785	0.789 ($\uparrow 0.51\%$)	0.781	0.802	0.820 ($\uparrow 4.99\%$)
	R@10	0.764	0.835 ($\uparrow 9.29\%$)	0.814	0.858 ($\uparrow 5.41\%$)	0.832	0.850 ($\uparrow 2.16\%$)	0.848	0.859	0.874 ($\uparrow 3.07\%$)

CCSD and CCHNR are short for CoCoSoDa and CoCoHaNeRe.

Method: To evaluate whether our method can be applied to different PTM, we chose RoBERTa, CodeBERT, GraphCodeBERT, and UniXCoder for experiments. Since CoCoSoDa also uses UniXCoder as the backbone encoder, we list its results. The most important technique of our model is to accurately identify similar examples, so we choose top-1, top-5, and top-10 as evaluation indicators. For different models, we use the same output method, that is, the classification embedding (CLS) output by the encoder.

Results: CoCoHaNeRe achieves the best results on all models. The improvement effect on RoBERTa is amazing, reaching 20% on the top-1 indicator. There is also a 10% improvement on the top-10 indicator. Our improvement effect on all models exceeds 5%. The best top-k effect is achieved on the UniXCoder model.

The improvement of different languages is distributed differently across various models. For example, our method demonstrates a very noticeable improvement on the JavaScript dataset with the RoBERTa model and the CodeBERT model. However, the improvement is not as apparent with the UniXCoder model. Additionally, the effect of our CodeBERT on the Ruby dataset even surpasses that of the UniXCoder model. However, the improvement effect on the Go language remains the smallest.

Overall, the improvement in the top-1 ranking is obviously greater than that in the top-5 ranking, and the improvement in the top-5 ranking is obviously greater than that in the top-10 ranking. This indicates that our method indeed prioritizes placing the correct results at the forefront.

Discussion: As is shown in Table 3, CoCoHaNeRe outperforms CoCoSoDa and achieves new SOTAs on all the adopted datasets. Besides, UniXCoder performs better than RoBERTa and CodeBERT when adapted to both CoCoSoDa and CoCoHaNeRe. It is reasonable since UniXCoder has the ability to incorporate both semantic and syntax information from code comments and ASTs by utilizing more powerful pre-training tasks, while RoBERTa and CodeBERT are pre-trained on fewer pre-training tasks.

5.3 Model Albation (RQ3)

Quick View of Answers and Implications

The BCL and HNCL loss in CoCoHaNeRe have different effects on the model, but they both improve the model's performance. HNCL contributes more to the improvement of the model's effectiveness. For the multi-modal hard negative sampling strategy, bi-modal selection is better than uni-modal. We recommend training with a combination of BCL and HNCL on new datasets, but their weights should vary according to their validation results.

Method: We conducted ablation on the BCL and HNCL to investigate their impacts on CoCoHaNeRe. We adapted UniXCoder to CoCoHaNeRe in the following experiments. Additionally, we examined the impacts of using different ways to search k nearest neighbors (knn) for query/code from the memory bank to construct hard negative examples. Uni-knn stands for searching for a similar query/code for query/code, while bi-knn stands for searching query/code for code/query. Ablation results are shown in Table 4.

Results: CoCoHaNeRe (-w/o HNCL) only uses BCL, and its effect is similar to the original model, which only improves by about 2%. For uni-knn that retrieves hard negative for a single modality, its effect is significantly lower than that of bi-knn for two modalities. Interestingly, in uni-knn, the effect of BCL is very obvious. The effect of CoCoHaNeRe (uni-knn -w/o BCL) is 3% lower than that of CoCoHaNeRe (uni-knn Full). The CoCoHaNeRe (bi-knn Full) with multiple losses and two modalities has the best effect.

Table 4. Results of Ablations on Different Loss Functions (BCL and HNCL) and Neighbor Sampling Strategies (uni-knn and bi-knn)

Methods		Java		Python		PHP		JS		Ruby		Go	
		MRR	R@1	MRR	R@1	MRR	R@1	MRR	R@1	MRR	R@1	MRR	R@1
bi-knn	Full(CoCoHaNeRe)	0.749	0.663	0.747	0.653	0.701	0.603	0.700	0.606	0.741	0.646	0.922	0.885
	-w/o BCL	0.733	0.645	0.726	0.632	0.686	0.588	0.667	0.576	0.713	0.620	0.914	0.875
uni-knn	Full	0.732	0.645	0.722	0.626	0.682	0.583	0.662	0.564	0.710	0.614	0.913	0.874
	-w/o BCL	0.708	0.616	0.722	0.627	0.681	0.579	0.667	0.572	0.710	0.614	0.912	0.871
	-w/o HNCL	0.691	0.593	0.696	0.595	0.648	0.542	0.639	0.539	0.678	0.570	0.898	0.850
	-w/o all	0.677	0.580	0.672	0.574	0.626	0.521	0.621	0.514	0.679	0.583	0.885	0.837

Table 5. Pre-Training Time Usage of Different k with MBS = 8,192 and BS = 32

	$k = 1$	$k = 3$	$k = 5$	$k = 7$	$k = 9$
Pre-training Time	1 h 5 m 48 s	1 h 29 m 36 s	1 h 56 m 30 s	2 h 26 m 35 s	2 h 53 m 36 s

Discussion: Comparing the results of UniXCoder with the loss of “*bi-knn Full*” and “*bi-knn -w/o BCL*” in Table 4, we find that there is apparently decrease when removing the “*BCL*” loss. But the Ruby dataset is an outlier which implies that the effect of the loss function may be influenced by the dataset. We recommend training with a combination of BCL and HNCL on new datasets, but their weights should vary according to their validation results.

When comparing the results of the “*bi-knn*” and “*uni-knn*,” we find that all unimodal retrieval methods performed significantly weaker than the bimodal method. This supports our hypothesis that our sampling strategy can improve the model’s ability to distinguish strong negative samples, and also suggests that the distribution of code embedding does not align perfectly with the distribution of text, making unimodal retrieval less effective.

For the *uni-knn* method, the results after removing the *BCL* loss still followed a similar pattern to the *bi-knn* method on datasets in different languages. Results for Full and -w/o *BCL* versions on the Ruby dataset were also very close. However, using *uni-knn* still significantly outperformed the results without the *HNCL* loss. This indicates that the *HNCL* loss has a much larger impact on the results and a more stable effect compared to the *BCL* loss. At the same time, this also suggests that PTM can more effectively align the representation distributions of code and query, allowing the unimodal method to achieve good results even when considering indirect training code and query similarities.

5.4 Impacts of Hyperparameters (RQ4)

Quick View of Answers and Implications

Hyperparameters have different impacts on the model performance depending on the size of the dataset, the encoder, and other hyperparameters. More hard negatives do not necessarily lead to better results. We recommend that users choose a k value with a smaller training cost based on the size of the dataset. Train on a small dataset first to determine the choice of other hyperparameters.

Method: To study the impact of different hyperparameters on our experimental results, we chose four hyperparameters for experiments, namely memory bank size, k , batch size, and learning rate. We conducted experiments under different hyperparameters and observed the changes in the

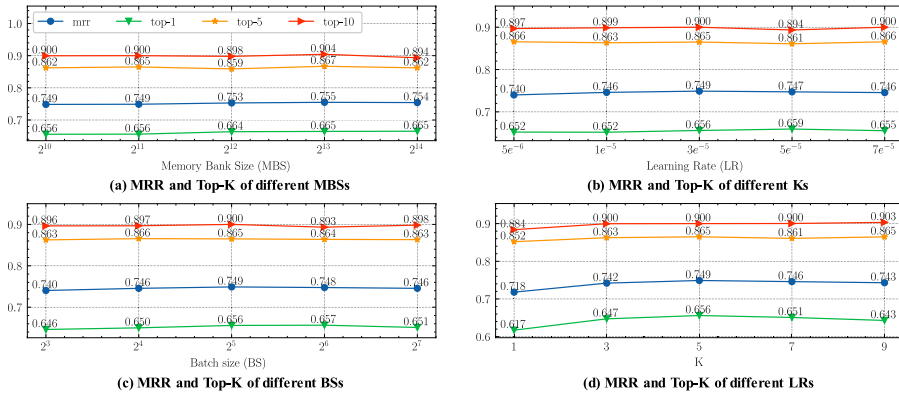


Fig. 7. Impacts of memory bank size, k , and batch size.

experimental results. We set the baseline value, that is, the k -nearest-neighbor selected each time is 5, the memory bank size is 8,192, the batch size is 32, and the learning rate is $3e^{-5}$. Each experiment only changes one dimension, and the others remain at the baseline value. Due to the limited conditions, we only choose 5 values for comparison. We chose MRR, top-1 precision, top-5 precision, and top-10 precision as evaluation indicators. We visualize the experimental results and then analyze them.

Because our method incorporates hard negatives into the model training process. Therefore, the k parameter that determines the number of hard negatives will affect the training speed of the model. We counted the time of model training under different k values.

Results: Experimental results are shown in Figure 7. CoCoHaNeRe performs best when the memory bank size is 2^{13} . When the learning rate is $3e^{-5}$ and $7e^{-5}$, the model performs best. When the k value becomes larger and larger, the model performance gradually increases, but the training cost of the model increases linearly. It takes about twelve minutes to increase the model training time by 1. When setting the batch size as 32, the model performs best.

Discussion: (1) *Memory Bank Size.* We investigated the impact of memory bank size on performance. As is common in previous work, we used power of 2 sizes to test. This allows us to quickly determine the optimal interval. We found that the memory bank does not always improve with increasing size. This is because the embedding features stored in the memory bank are generated by the encoder at different epochs, leading to differences in the distribution of these embeddings. The CoCoSoDa method addresses this issue by using a momentum encoder to slow down the model update, resulting in more slowly changing embeddings in the memory bank. We recommend adjusting the size of the memory bank based on the size of the dataset when using the CoCoHaNeRe algorithm on a new dataset.

(2) *k in k Nearest Neighbors.* We found that larger values of k did not consistently lead to better performance for our algorithm. This may be due to feature drift caused by model learning, similar to the issue with the memory bank size. Our algorithm relies on differentiable embedding for negative examples, which requires recalculating the nearest neighbors for code or text each time, leading to slower performance compared to traditional methods and a larger required batch size. However, this can be addressed by reducing the number of examples per computation of the neighbor contrastive loss. Our algorithm also requires more memory and GPU memory compared to using a momentum encoder, although the GPU memory consumption is still relatively small. Additionally, our model may be slower in large datasets due to the need to search for k nearest neighbors for each example, and the size of the memory bank may need to be set very large in

sparse data, increasing computational cost. However, the cost of calculating k nearest neighbors is relatively low compared to the computational cost of complex PTM.

With the increase of k , the training cost of the model increases linearly in Table 5. When $k = 1$, the training cost of the model is the lowest, but the effect of the model is also the worst. When $k = 5$, the training cost and effect of the model have reached a balance. When $k = 9$, the training cost of the model is the highest, but the effect of the model has not reached the best. This shows that the size of k has an impact on the effect of the model, but the size of k is not the bigger the better. We recommend choosing a smaller k value according to the size of the dataset when using our algorithm on a new dataset to achieve better results.

It is important to note that the search speed of our model is equivalent to that of other models. This is because, in production environments, we first employ the model to generate embeddings for text or code snippets, which are then stored. During the search process, we directly calculate the similarity between these pre-computed embeddings. The inference speed of the model solely depends on the model's parameters and the hardware environment, and is not influenced by our algorithm.

(3) *Batch Size*. For traditional tuning algorithms, it is typically beneficial to use a larger batch size. However, in our experiments, we found that increasing the batch size led to faster updates of the embeddings in the memory bank and a decrease in performance. We recommend using a batch size of 32 or 64 for training to achieve optimal results.

(4) *Learning Rate*. We found that the learning rate did not have a significant impact on results. We recommend using a learning rate of $3e^{-5}$ or $7e^{-5}$ for training to achieve optimal results.

5.5 Applying to Other Code Retrieval Tasks (RQ5)

Quick View of Answers and Implications

Our model also achieved superior results in code clone detection and code question answering tasks. This shows that the features learned by our model can be transferred to other code understanding tasks. And it shows that the hard negative problem exists in multiple tasks.

We recommend using our method to train the model on large code and text datasets and fine-tuning the original contrastive learning method on downstream tasks to transfer the features learned by our model to other tasks.

Method: There are several tasks in code understanding that are related to matching code and text. These tasks have many similarities to code search tasks, such as code clone detection, code question answering, etc. Therefore, we apply our method to these tasks to verify whether our method can be applied to other code retrieval tasks. We conducted experiments on POJ-104, BigCloneBench, and CoSQA. We follow the settings in CodeXGLUE, first pre-training our model on CSN, and then fine-tuning the model on the downstream task dataset. We use MAP@R, Recall, Precision, and F1-score as evaluation metrics for clone detection. We use Accuracy as the evaluation metric for code question answering.

The CoSQA dataset is collected from the search logs of the Microsoft Bing search engine. Consequently, it bears significant similarity to datasets for code retrieval, as both involve matching text and code based on their similarities. Therefore, we believe that our proposed model can yield performance improvements on this benchmark. Code clone detection primarily focuses on examining the similarity between different code snippets. The POJ-104 dataset consists of similar C language code collected from an online judge, while BigCloneBench filters out identical code

Table 6. Results on Other Retrieval Tasks: Code Question Answering and Clone Detection

Model	Clone Detection				Code Question Answering
	POJ-104	BigCloneBench			CosQA
	MAP@R	Recall	Precision	F1-score	Accuracy
RoBERTa	76.67	95.1	87.8	91.3	60.3
CodeBERT	82.67	94.7	93.4	94.1	65.7
GraphCodeBERT	85.16	94.8	95.2	95.0	68.4
SynCoBERT	88.24	-	-	-	-
PLBART	86.27	94.8	92.5	93.6	65.0
CodeT5	88.65	94.8	94.7	95.0	67.8
CodeT5+	90.12	95.1	94.1	95.2	68.3
UniXcoder	90.52	92.9	97.6	95.2	70.1
CoCoSoDa	91.29	94.8	95.2	95.0	75.0
CoCoHaNeRe	91.74	95.2	96.2	95.4	75.2

implementations from Java datasets. Since code clones may require identifying subtle differences between different code snippets, it is evident that learning hard negatives is essential. Consequently, we believe that our model can also be effective in addressing this task.

Results: Experimental results are shown in Table 6. Our model outperforms other baseline models on POJ-104 and CoSQA. In the results of BigCloneBench, both our Recall and F1 exceed other models. The Precision indicator did not exceed UniXCoder, but it was close. Our model performs best on the CoSQA dataset, with an improvement of 5% over the backbone CodeT5+.

Discussion: Our model's performance on CoSQA shows that our method can learn the matching information between code and text well, and these features can be applied to the code question answering task. We found that different from code search, the encoder-decoder architecture model (PLBART and CodeT5) in code question answering and code clone detection performs no worse than other encoder architecture models. Perhaps using the encoder-decoder architecture model to test these two tasks, the effect will be better.

The results on BigCloneBench show that code clone detection is still slightly different from code search tasks. Overall, the improvement achieved in code clone detection is relatively small. We analyze the reasons as follows: ① Firstly, our model learns the matching between text and code during pre-training, while code clone datasets involve matching between pieces of code. This difference may lead to a decrease in model performance. ② The POJ-104 dataset is relatively small and consists of C language data, while our model has not been pre-trained on C language datasets, thus lacking relevant knowledge. ③ Existing methods have already achieved high levels of accuracy, such as unixcoder, which has reached 97.6%.

We believe the following measures are needed to improve performance in code clone detection tasks: ❶ We need larger datasets for pre-training. We propose expanding existing datasets using techniques such as code fuzzing. ❷ Learning should be conducted on datasets encompassing multiple PLs. ❸ During pre-training, incorporating uni-modal similarity contrastive learning loss tailored to code-to-code similarity can enhance the model's ability to recognize similarity between pieces of code. Finally, we believe that existing PTM may not be the best option for code clone detection tasks. Since PTM tend to focus more on the semantics of text, whereas clone detection may require attention to the structure of code, using graph neural networks and outputting structural information such as AST and CFG may be better improvement measures.

Table 7. The Performance of Different Approaches on XLCoST Dataset Evaluated by MRR Scores

Model	C++	Java	Py	C#	JS	PHP	C
RoBERTa	0.5034	0.4515	0.5001	0.5249	0.5039	0.5561	0.5272
CodeBERT	0.5018	0.4981	0.495	0.5216	0.5037	0.5216	0.5823
GraphCodeBERT	0.5059	0.5222	0.4968	0.5094	0.4956	0.531	0.5854
UniXCoder	0.5557	0.5605	0.5305	0.5538	0.5186	0.5508	0.6203
CoCoSoDa	0.5906	0.5933	0.5684	0.5891	0.5742	0.6863	0.7065
CoCoHaNeRe	0.6159 ↑4.28%	0.6338 ↑6.83%	0.6201 ↑9.1%	0.6315 ↑7.2%	0.6219 ↑8.31%	0.6875 ↑0.17%	0.7483 ↑5.92%

5.6 Performance of CoCoHaNeRe with Real World Dataset (RQ6)

Quick View of Answers and Implications

Because CSN uses user comments to simulate code search, it cannot simulate real-world scenarios with a lot of redundant data. We used the XLCoST dataset, which represents real-world scenarios, to validate if our method is applicable to real code retrieval scenarios. The results show that our method achieves significant improvements, even on datasets in languages not encountered during pre-training. This indicates that our method is applicable to real-world scenarios.

Method: Using the same setup as in code question answering and clone detection, we first pretrain our method for one epoch on the CSN dataset, and then fine-tune it on the monolingual dataset of the XLCoST dataset. Early stop mechanism is employed during fine-tuning. Evaluation on XLCoST also utilizes the MRR metric. We train using the code provided by XLCoST. It's worth noting that we did not use CodeT5+ as the backbone model here, as we found that the performance of the CodeT5+ model on XLCoST was not satisfactory. Therefore, we opted for the UniXCoder model as the backbone.

Results: The results of different models on the real dataset XLCoST are shown in Table 7. In terms of experimental methodology, our approach still achieves the best performance. The improvement achieved on the Python language reaches up to 9% at maximum. We found that our model still achieved significant improvements on datasets of languages not seen in previous pre-training. For example, it achieved gains of 7.2% and 5.92% on C# and C, respectively.

Discussion: The XLCoST dataset collects real-world code knowledge questions and answers, thus it can be used to simulate the characteristics of real data. In real code search scenarios, there is a lot of redundant information, such as a large amount of language description by users that is not used to describe the semantics of the code, but rather their environment configuration, development experience, etc. Other users' answers also contain some discussions, which can be considered redundant information. Our model has learned these features well and has achieved significant improvement, indicating that our approach can help the model be applied to real scenarios. We believe that the reason our model has improved on languages it has not seen in pretraining may be because C language and Go language are somewhat similar, and C# language and Java language are also quite similar, so our model has learned similar features.

Table 8. The Performance of Different Approaches under the Zero-Shot Experimental Setting Evaluated by MRR Scores

Model	Ruby	JS	Go	Python	Java	Php	Avg
CodeBERT	0.002	0.001	0.002	0.001	0.001	0.001	0.001
CodeT5	0.006	0.003	0.009	0.001	0.001	0.001	0.004
GraphCodeBERT	0.238	0.111	0.209	0.137	0.123	0.120	0.156
CodeT5+	0.613	0.494	0.701	0.506	0.522	0.436	0.543
UniXCoder	0.576	0.442	0.648	0.447	0.466	0.373	0.492
CoCoSoDa	0.706	0.609	0.831	0.642	0.693	0.599	0.680
CoCoHaNeRe	0.741 ↑4.96%	0.662 ↑8.7%	0.910 ↑9.51%	0.712 ↑10.9%	0.730 ↑5.34%	0.675 ↑12.69%	0.738 ↑8.57%

Query Text: Fire notification events within the current game state. Wrong Code: def notification(event, sender=nil) sender = sender UnknownSender state.fire_events_for_notification(event, sender) End	Correct Code: def fire_events_for_notification(event, sender) current_state.each { cs cs.fire_events_for_notification(event, sender) } End
--	--

Fig. 8. Example of related Code call.

5.7 Performance of CoCoHaNeRe without Being Fine-Tuned (RQ7)

Quick View of Answers and Implications

CoCoHaNeRe's results on the test set without fine-tuning are much better than other models, and even exceed the effect of other PTM fine-tuning on the downstream dataset. This shows that our method can be well applied to low-resource scenarios. We recommend that if users have insufficient data or poor data quality, they can try the zero-shot method to use the model.

Method: As real-world scenarios are often low-resource, that is, users lack a sufficient amount of effective data for fine-tuning. Therefore, the model needs to make predictions directly on the test set without fine-tuning, which is called zero-shot. We conducted zero-shot experiments on CSN, that is, training on the training set and then predicting on the test set without fine-tuning on the single-language dataset. We compared CoCoHaNeRe with other PTM, including CodeBERT, CodeT5, GraphCodeBERT, UniXCoder, and CoCoSoDa. We use MRR as the evaluation metric.

Results: Experimental result on CSN is shown in Table 8. It shows that CoCoHaNeRe outperforms other models on the test set of all languages. In terms of average MRR, CoCoHaNeRe is 7.30% higher than CoCoSoDa. At the same time, models without code pre-training, such as CodeBERT and CodeT5, have almost no ability to predict.

Discussion: We can see that the performance of models without pretraining is poor. But CoCoHaNeRe, which has undergone contrastive learning and KNN sample training, learns a more unified single-modal sample distribution and a better alignment distribution of bimodal data. Therefore, CoCoHaNeRe exceeds other models by 50%. Furthermore, our zero-shot performance even exceeds the performance of other models after fine-tuning.

Query Text: Retrieve and return the commit at which the provided fork was forked at. Wrong Code: <pre>def retrieve_commit(repo, sha, user) commit = persister.find(:commits, {'sha' => "#{sha}"}) if commit.empty? c = api_request(url) ... persister.store(:commits, c) else debug "Commit #{user}/#{repo} -> #{sha} exists" commit.first end end</pre>	Correct Code: <pre>def ensure_fork_point(owner, repo) fork = ensure_repo(owner, repo, false) # Return commit if already specified unless fork[:forked_commit_id].nil? commit = db[:commits].where(:id => fork[:forked_commit_id]).first return commit unless commit.nil? end ... forked_commit = ensure_commit(repo, forked_sha, owner); unless forked_commit.nil? db[:projects].filter(:id => fork[:id]).update(:forked_commit_id => forked_commit[:id]) end db[:commits].where(:sha => forked_sha).first end</pre>
--	--

Fig. 9. Example of information at the back of the text or code been overlooked.

Query Text: Fire notification events within the current game state. Wrong Code: <pre>def do_json_request(method, url, params = nil, additional_headers = {}, &create_response) begin ... response = do_request(method, url, (params.to_json if params), additional_headers) rescue RestClient::Exception => e error_details = e.http_body ? JSON.parse(e.http_body) : nil ... end end</pre>	Correct Code: <pre>def get(url) request = Net::HTTP::Get.new(uri.request_uri, @headers) response = https.request(request) if response.code != '200' check_response(response.code, response.body) else parse_json(response.body) end end</pre>
--	---

Fig. 10. Example of text description describing parameter and return value information.

5.8 Qualitative Analysis

We perform qualitative analysis of the test results of our approach. By analyzing the cases of search results, we find that the cases can be divided into the following situations.

5.8.1 Code Related to the Correct Function. As shown in the case 2 of Figure 8, there is a wrong code snippet that calls the correct function match the query, so the baseline PTM cannot distinguish it from the correct function. The correct function here is “*fire_events_for_notification*”, called in this wrong code snippet. Likewise, traditional methods cannot tell because it was not sampled in the same batch during training. But our model can recognize this error.

5.8.2 Information at the Back of the Text or Code Is Easily Overlooked. The example given in Figure 9 illustrates that traditional models tend to focus on the information in the front of the text description. In the example, “*retrieve*” and “*commit*” appear first, but “*fork*” information is equally important. Our model successfully matched the code containing the fork recognition function. This also indicates that the order of text description is equally important, and simply increasing data and model size cannot make the model pay attention to the information contained in the order of language.

5.8.3 Text Descriptions Contain Information about Function Parameters and Return Values. In text descriptions, there are often requirements for function parameters and return values. However, traditional models can only pay attention to words that are similar to parameters and return values, without distinguishing their positions. In the example in Figure 10, the code that CoCoSoDa misidentified contained words similar to those in the text description, such as “*parse*,” “*request*,” and

Query Text: Adds a callback that records a version after a "create" event. Wrong Code: <pre>def record_destroy(recording_order) return unless enabled? && !@record.new_record? in_after_callback = recording_order == "after" event = Events::Destroy.new(@record, in_after_callback) # Merge data from 'Event' with data from PT-AT. We no longer use # `data_for_destroy` but PT-AT still does. data = event.data.merge(data_for_destroy) version = @record.class.paper_trail. version_class.create(data) if version.errors.any? log_version_errors(version, :destroy) else assign_and_reset_version_association(version) end end</pre>	Correct Code: <pre>def get(url) request = Net::HTTP::Get.new(uri.request_uri, @headers) response = https.request(request) if response.code != '200' check_response(response.code, response.body) else parse_json(response.body) end end</pre>
---	---

Fig. 11. Example of text description more similar to wrong code.

“json.” However, these are not return values or parameters. Our model successfully matched the correct code. This indicates that parameters and return values are also related to their positions and cannot be matched using simple word similarity.

5.8.4 Text Description More Similar to Wrong Code. As shown in Figure 11, recognizing the semantics of code requires more than just lexical similarity. Compared to correct code, CoCoSoDa’s incorrect identification of wrong code is more vocabulary similar to text descriptions. However, the functionality implemented by the erroneous code is exactly opposite to the create event required by the text description.

6 Related Work

6.1 Code Pre-Trained Model

Pre-training refers to training models on a large dataset using unsupervised training tasks (i.e., pre-training tasks) that treat the data itself as labels. Models derived by pre-training are PTM. Afterwards, PTMs are supervisedly trained on the small datasets for the downstream tasks, and this process is called fine-tuning. Owing to the ability of pre-training in encoding general linguistic and commonsense knowledge about language, the fine-tuned models can achieve better results on downstream tasks.

A significant number of PTM specifically designed for source code (CodePTMs) [9, 38, 39, 40, 48, 54, 75] have been successfully developed in recent years and they can be divided into two groups. The first group are models from NLP domain but re-pre-trained on source code, such as CuBERT, CodeGPT-adapted [51], and DeepDebug [19], etc. CuBERT is based on BERT [17] (whose backbone is Transformer [77] Encoder) where the basic MLM and next sentence prediction pre-training tasks are utilized. CodeGPT-adapted is derived by pre-training the GPT-2 [67] (based on the Transformer Decoder) on source code and the pre-training task is unidirectional language model. DeepDebug is an adaption of T5 [68] (a complete Transformer) on source code whose pre-training task is a sequence-to-sequence version of MLM. The second group are models specially designed by considering characteristics of source code compared to NL, such as CodeBERT [21], GraphCodeBERT [26], T5-learning [82], and PLBART [5]. CodeBERT and GraphCodeBERT are based on RoBERTa [49] designed by pioneers in the field of NLP. Different from RoBERTa, CodeBERT adopts both code and corresponding NL description as input, while GraphCodeBERT adopts Data

Flow Graph in addition to code and NL. T5-learning and PLBART are based on T5 and BART [44], respectively, and both of them utilize code and NL as the input.

According to Niu et. al. [61], the SOTA results of 18 SE tasks on 30 corresponding datasets are achieved by CodePTMs, which provides suggestive evidence that CodePTMs are a promising approach to a wide variety of SE tasks. In our experiments, we will use representative PTM that have been applied to code search as baselines, and they are aforementioned RoBERTa, CodeBERT, and GraphCodeBERT.

6.2 Contrastive Learning

Contrastive learning [37] has become a new research trend of more effectively solving classification problems. It gets rid of the limitation of supervised loss function (e.g., Cross Entropy Loss [91]) based on classification label, directly regards matched examples as positive examples, and unmatched examples as negative examples. It learns feature mapping by decreasing the distance between positive examples and increasing the distance between negative examples. Several works [8, 20] attempt to use contrastive learning on NL and PL. Fang et al. [20] proposed CERT, treating one translated sentence and original sentence as a positive pair. Bui et al. [8] exploited contrastive learning on PLs. They trained a neural network to compare similar and unrelated code snippets with a contrastive learning objective. However, they overlooked the multi-modal characteristic of PLs.

Contrastive learning requires a large number of negative examples for comparison. Therefore, contrastive learning methods in computer vision use data augmentation methods to generate different positive examples for one image [12, 57, 76]. However, for sequence data such as code or text, study [23] has shown that using data augmentation is not the best method, simply dropout can outperform those augmentation algorithms.

6.3 Hard Negative Mining for Contrastive Learning

Hard negative samples are those that closely resemble positive samples, making them challenging to differentiate. The practice of mining hard negatives is widespread in contrastive learning, with numerous studies across various domains. Many of these studies focus on multi-modal *computer vision* tasks [15, 33, 47, 65, 66]. For instance, Huang et al. [33] introduced Structure-CLIP to develop multi-modal structured representations. They generated hard negative captions through keyword exchange and calculated loss based on the distance between these hard negatives and the authentic captions. Radenovic et al. [65] employed in-batch hard negative mining to enhance image retrieval performance, assigning different weights to hard and easy negatives in the loss function. Our approach differs from these works as we don't use in-batch negative mining. Instead, we consider the entire dataset, and our hard negatives are actual samples from the dataset rather than augmented data.

Hard negative mining has been applied in other fields as well, though not significantly differently from the aforementioned works. In *audio-text retrieval*, Xie et al. proposed 8 methods for sampling hard negatives [85], but these are still in-batch methods based on various similarity metrics. In *graph contrastive learning*, CuCo [16] arranges negatives from easy to hard based on similarity for graph-level contrastive learning and suggests automatically selecting and training negative samples using curriculum learning. Xia et al. [84] developed ProGCL to estimate the likelihood of a negative being genuine, providing a more suitable measure of negative hardness along with similarity. In the *recommendation system* domain, SRNS [18] proposed a variance-based sampling function using observed statistical features to identify hard negative samples. MixGCF [32] designed hop mixing and positive mixing strategies to synthesize informative hard negatives. AugNS [88] achieves hard

negative augmentation by introducing uniform noise to the representation, preserving most of the original information while introducing semantic differences.

In *dense information retrieval*, contrastive learning methods have also been employed. Some approaches use random sampling to select negative samples. For example, Huang et al. [30] used random negative sampling to approximate the recall task. Karpukhin et al. [41] and Zhan et al. [90] adopted in-batch training, using relevant documents from other queries in the same mini-batch as negatives. Some methods have addressed the hard-negative problem. Gao et al. [22] and Karpukhin et al. [41] used BM top documents as hard negatives, an offline method that differs from our continuous selection of hard negatives during training. Xiong et al. [86] proposed Approximate Nearest Neighbor Negative Contrastive Estimation, which searches for negative examples in the global database and continuously updates each sample's approximate nearest neighbor index during training. Zhan et al. [89] discussed selection methods for dynamic and static hard negatives and proposed two training algorithms combining strong negatives.

Our method differs in several ways. First, we dynamically update the embedding of hard negatives during training, rather than generating them statically. Second, due to the need for dynamic updating, our method requires more memory, which is why we limit our calculations to a maximum of ten hard negatives. In contrast, Zhan et al. [89] use 200 hard negatives, and CoCoSoDa [73] uses 4096 negative samples, as their hard negatives don't participate in model calculations and thus don't occupy memory. Importantly, our paper focuses not on obtaining hard negatives from the database but on how hard negative samples participate in the gradient descent process of fine-tuning PTM. Third, our method applies to the code retrieval task, not document retrieval. Code retrieval has unique characteristics: (1) similarity between code and text, (2) similar codes may have entirely different semantics, and (3) the prevalence of similar codes in codebases (e.g., due to calls and inheritance) makes hard negatives more crucial. In dense retrieval, document retrieval doesn't require a high top-1 index. Particularly in image-text search, the search engine only needs to find an image matching the description, not a specific image.

7 Discussion

7.1 Applicable Discussion

Limitations. Although the toolname has achieved a good effect improvement, our model still produces incorrect matches. After sorting out the error samples, we found that the model mainly produces errors for the following 5 examples:

- (1) Text descriptions or code lengths are too long.
- (2) The code contains knowledge such as APIs or command lines.
- (3) The code found by the model also matches the text description.
- (4) Non-standard noise, such as a large number of useless comments or data.
- (5) The text is not semantically targeted at the code, but at project development.

An example of the second case is shown in Figure 13. The correct code invokes the command line tool "cocaine." The code search engine needs to understand the command line input and output format of "cocaine" in order to correctly retrieve the code. An example of the fifth case is shown in Figure 12. The comments provided by the programmer here serve as reminders of the function's role in project development, facilitating subsequent development and debugging. However, the functionality of the function itself is not described. The incorrect code contains vocabulary similar to the "query," resulting in incorrect matching. Other cases are relatively straightforward, so we will not show examples here. Based on these results, we believe future code retrieval models need to consider dataset cleaning and learning more about API and project knowledge.

Query Text: for manual use, maybe when migrating	Correct Code: <pre>def get_network_by_name(organization, networkName) result = nil organization[:networks].each do network if network[0].downcase == networkName.downcase result = get_network(network[1]) end end result end</pre>
Wrong Code: <pre>def migrate(row, migrated) super self.site = Galena.administrative_objects.tissue_bank self.storage_type = Galena.administrative_objects.box_type end</pre>	

Fig. 12. Example of text description describing project development knowledge but not program semantics.

Query Text: Parses options and converts them to Cocaine syntax, return commands ready to do cocaine	Correct Code: <pre>def options_to_commands commands = [] @options.sanitize_keys.each_paramized_key do key, paramized_key if @options[key].to_s == 'true' commands.push "--#{paramized_key}" elsif @options[key].to_s == 'false' commands.push "--no-#{paramized_key}" else commands.push "--#{paramized_key} :#{key}" end end commands.push quoted(url) commands.join(' ') end</pre>
Wrong Code: <pre>def parse(args) @options = Options.new opt_parser = OptionParser.new do parser @options.define_options(parser) end opt_parser.parse!(args) @options end</pre>	

Fig. 13. Example of code contains knowledge such as APIs or command lines.

Ideal Application Scenario. CoCoHaNeRe is well-suited for scenarios involving a substantial amount of redundant data in code retrieval tasks. For instance, in large-scale code repositories of companies, there are often numerous similar components, and code utilizing inheritance or polymorphism may generate a significant quantity of redundant code. Distinguishing the semantics of different codes using traditional methods can be challenging in such cases. However, for small, local code repositories, the zero-shot model can be employed for search purposes without the need for our proposed model. Regarding large network code repositories, we believe that the efficiency of current dense retrieval methods may not be practical for application at present. Nevertheless, traditional search engines can be utilized initially, and subsequently, the results can be re-ranked using our proposed model.

The dataset redundancy can be defined as follows:

$$R = \frac{N_{high-sim}}{N_{total}}. \quad (12)$$

In this equation, R represents the redundancy of the dataset, $N_{high-sim}$ denotes the number of text pairs with high similarity and N_{total} denotes the total number of text pairs. The similarity of texts needs to be defined by the user, who is required to establish a threshold for highly similar text pairs based on experience. Common methods for measuring similarity include cosine similarity, Jaccard similarity [35], and edit distance, among others. We recommend that users initially employ simple methods, such as **Naive Bayes on Bag-of-Words (NBOW)** [53], to assess the redundancy in their dataset. For datasets with low redundancy, standard methods should suffice. However, for datasets with high redundancy, our method may be needed to identify the differences between similar text pairs in order to learn subtle features.

Practice Suggestion. Before using our model, we recommend cleaning and filtering the data. First, use simple methods like NBOW to remove comments that are not very similar. For code without any comments, use comment generation models to create comments. For long code, use program slicing techniques [83] to identify and remove unimportant parts of the code.

7.2 Threats to Validity

Construct Validity. Threats to construct validity are associated with the evaluation metrics employed. For instance, we can utilize the NDCG metric [81] to evaluate scenarios where multiple correct answers exist within the dataset. Additionally, we adopt statistical significance testing to measure the differences between our implementation and the original ones.

Internal Validity. Internal validity threats often stem from hyperparameters in deep learning models. In our experiments, we maintain consistency by refraining from altering hyperparameters, although adjusting certain ones could potentially enhance performance.

External Validity. Our comparison is limited to the RoBERTa and T5 architecture models, neglecting other models like the GPT architecture. The performance of our algorithm on these alternative models requires further verification. Secondly, some models have achieved good results by utilizing a decoder for rearrangement. We aim to verify the performance of our model after incorporating decoder rearrangement.

8 Conclusion and Future Work

In this paper, we propose CoCoHaNeRe, an augmented contrastive learning-based code search framework utilizing a specially designed memory bank. For each batch, only k nearest codes/queries of each query/code are retrieved from the memory bank to construct real hard negative examples. The retrieved queries/code will also go through the encoder to participate in the gradient descent. Besides, the memory bank is continuously updated after handling each batch. Experimental results demonstrate (1) the proposed CoCoHaNeRe surpasses current SOTA approaches in adapting all of the evaluated code pre-training models, (2) new SOTA of code search on the CSN is CoCoHaNeRe adopting CodeT5+, (3) both the bimodal and HNCLs contribute to CoCoHaNeRe, and (4) bi-modal searching on the memory bank is more effective than uni-model searching. Our replication package can be found at [3].

In the future, we plan to conduct more detailed research to investigate which kind of hard negative examples can help the model learn better representations and which kind is invalid. Secondly, we hope to apply our method to more fields, such as API recommendation and retrieval augmented generation. Thirdly, we can combine code translation technology to construct more datasets of PLs, so that our model can be applied to different PLs. The existing code search datasets are not large enough and contain a large amount of noisy data. We can try to construct larger datasets and train them with the latest large language models, which should achieve better results. Finally, we hope to use successful techniques in the field of large language models, such as prompt strategies, so that our model can retrieve according to the user's needs, such as identifying different categories of code, rather than just matching based on text and semantic similarity.

References

- [1] Facebook Website. 2023. <https://www.facebook.com>
- [2] Github Website. 2023. <https://www.github.com>
- [3] Our Replication Package. 2023. <https://bitbucket.org/codeintelli/cocohanere>
- [4] StackOverflow Website. 2023. <https://stackoverflow.com/>
- [5] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '21)*. Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.), ACL, 2655–2668. DOI: <https://doi.org/10.18653/V1/2021.NAACL-MAIN.211>
- [6] Shushan Arakelyan, Anna Hakhverdyan, Miltiadis Allamanis, Luis Garcia, Christophe Hauser, and Xiang Ren. 2022. NS3: Neuro-Symbolic Semantic Code Search. Retrieved from http://papers.nips.cc/paper_files/paper/2022/hash/43f5f6c5cb333115914c8448b8506411-Abstract-Conference.html

- [7] Joel Brandt, Philip J. Guo, Joel Lewenstein, Mira Dontcheva, and Scott R. Klemmer. 2009. Two studies of opportunistic programming: Interleaving web foraging, learning, and writing code. In *Proceedings of the 27th International Conference on Human Factors in Computing Systems (CHI '09)*. Dan R. Olsen Jr., Richard B. Arthur, Ken Hinckley, Meredith Ringel Morris, Scott E. Hudson, and Saul Greenberg (Eds.), ACM, New York, NY, 1589–1598. DOI: <https://doi.org/10.1145/1518701.1518944>
- [8] Nghi D. Q. Bui, Yijun Yu, and Lingxiao Jiang. 2021. Self-supervised contrastive learning for code retrieval and summarization via semantic-preserving transformations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*. Fernando Diaz, Chirag Shah, Torsten Suel, Pablo Castells, Rosie Jones, and Tetsuya Sakai (Eds.), ACM, New York, NY, 511–521. DOI: <https://doi.org/10.1145/3404835.3462840>
- [9] Luca Buratti, Saurabh Pujar, Mihaela A. Bornea, J. Scott McCarley, Yunhui Zheng, Gaetano Rossiello, Alessandro Morari, Jim Laredo, Veronika Thost, Yufan Zhuang, and Giacomo Domeniconi. 2020. Exploring software naturalness through neural language models. arXiv:2006.12641. Retrieved from <https://arxiv.org/abs/2006.12641>
- [10] José Cambronero, Hongyu Li, Seohyun Kim, Koushik Sen, and Satish Chandra. 2019. When deep learning met code search. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE '19)*. Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo (Eds.), ACM, New York, NY, 964–974. DOI: <https://doi.org/10.1145/3338906.3340458>
- [11] Brock Angus Campbell and Christoph Treude. 2017. NLP2Code: Code snippet content assist via natural language tasks. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME '17)*. IEEE Computer Society, 628–632. DOI: <https://doi.org/10.1109/ICSME.2017.56>
- [12] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. 2020. Unsupervised learning of visual features by contrasting cluster assignments. In *Proceedings of the Annual Conference on Neural Information Processing Systems 2020 (NeurIPS '20)*. Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.), Retrieved from <https://proceedings.neurips.cc/paper/2020/hash/70feb62b69f16e0238f741fab228fec2-Abstract.html>
- [13] Yitian Chai, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Cross-domain deep code search with meta learning. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 487–498. DOI: <https://doi.org/10.1145/3510003.3510125>
- [14] Wing-Kwan Chan, Hong Cheng, and David Lo. 2012. Searching connected API subgraph via text phrases. In *Proceedings of the 20th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-20)*. Will Tracz, Martin P. Robillard, and Tefvik Bultan (Eds.), ACM, New York, NY, 10. DOI: <https://doi.org/10.1145/2393596.2393606>
- [15] Yen-Chun Chen, Linjie Li, Licheng Yu, Ahmed El Kholy, Faisal Ahmed, Zhe Gan, Yu Cheng, and Jingjing Liu. 2020. Uniter: Universal image-text representation learning. In *Proceedings of the European Conference on Computer Vision*. Springer, 104–120.
- [16] Guanyi Chu, Xiao Wang, Chuan Shi, and Xunqiang Jiang. 2021. CuCo: Graph representation with curriculum contrastive learning. In *Proceedings of the 13th International Joint Conference on Artificial Intelligence (IJCAI '21)*, 2300–2306.
- [17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '19)*. Jill Burstein, Christy Doran, and Thamar Solorio (Eds.), ACL, 4171–4186. DOI: <https://doi.org/10.18653/V1/N19-1423>
- [18] Jingtao Ding, Yuhan Quan, Quanming Yao, Yong Li, and Depeng Jin. 2020. Simplify and robustify negative sampling for implicit collaborative filtering. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 1094–1105.
- [19] Dawn Drain, Colin B. Clement, Guillermo Serrato, and Neel Sundaresan. 2021. DeepDebug: Fixing python bugs using stack traces, backtranslation, and code skeletons. arXiv:2105.09352. Retrieved from <https://arxiv.org/abs/2105.09352>
- [20] Hongchao Fang and Pengtao Xie. 2020. CERT: Contrastive self-supervised learning for language understanding. arXiv:2005.12766. Retrieved from <https://arxiv.org/abs/2005.12766>
- [21] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Proceedings of the International Conference on Findings of the Association for Computational Linguistics (EMNLP '20)*. Trevor Cohn, Yulan He, and Yang Liu (Eds.), ACL, 1536–1547. DOI: <https://doi.org/10.18653/V1/2020.FINDINGS-EMNLP.139>
- [22] Luyu Gao, Zhuyun Dai, Zhen Fan, and Jamie Callan. 2020. Complementing lexical retrieval with semantic residual embedding. arXiv:2004.13969. DOI: <https://arxiv.org/abs/2004.13969>
- [23] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Marie-Francine

- Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), ACL, 6894–6910. DOI : <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.552>
- [24] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)* Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.), ACM, New York, NY, 933–944. DOI : <https://doi.org/10.1145/3180155.3180167>
- [25] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL '22)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), ACL, 7212–7225. DOI : <https://doi.org/10.18653/V1/2022.ACL-LONG.499>
- [26] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of the 9th International Conference on Learning Representations (ICLR '21)*. OpenReview.net. Retrieved from <https://openreview.net/forum?id=jLoC4ez43PZ>
- [27] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross B. Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *Retrieved from IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '20)*. Computer Vision Foundation/IEEE, 9726–9735. DOI : <https://doi.org/10.1109/CVPR42600.2020.00975>
- [28] Emily Hill, Lori L. Pollock, and K. Vijay-Shanker. 2009. Automatically capturing source code context of NL-queries for software maintenance and reuse. In *Proceedings of the 31st International Conference on Software Engineering (ICSE '09)*. IEEE, 232–242. DOI : <https://doi.org/10.1109/ICSE.2009.5070524>
- [29] Reid Holmes, Rylan Cottrell, Robert J. Walker, and Jörg Denzinger. 2009. The end-to-end use of source code examples: An exploratory study. In *Proceedings of the 25th IEEE International Conference on Software Maintenance (ICSM '09)*. IEEE Computer Society, 555–558. DOI : <https://doi.org/10.1109/ICSM.2009.5306387>
- [30] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based retrieval in Facebook search. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '20)*. Rajesh Gupta, Yan Liu, Jiliang Tang, and B. Aditya Prakash (Eds.), ACM, New York, NY, 2553–2561. DOI : <https://doi.org/10.1145/3394486.3403305>
- [31] Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. 2021b. CoSQA: 20, 000+ Web queries for code search and question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP '21)*. Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.), ACL, 5690–5700. DOI : <https://doi.org/10.18653/V1/2021.ACL-LONG.442>
- [32] Tinglin Huang, Yuxiao Dong, Ming Ding, Zhen Yang, Wenzheng Feng, Xinyu Wang, and Jie Tang. 2021a. Mixgcf: An improved training method for graph neural network-based recommender systems. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 665–674.
- [33] Yufeng Huang, Jiji Tang, Zhuo Chen, Rongsheng Zhang, Xinfeng Zhang, Weijie Chen, Zeng Zhao, Zhou Zhao, Tangjie Lv, Zhipeng Hu, et al. 2024. Structure-CLIP: Towards scene graph knowledge to enhance multi-modal structured representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 2417–2425.
- [34] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436. Retrieved from <http://arxiv.org/abs/1909.09436>
- [35] Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Societe Vaudoise des Sciences Naturelles* 37 (1901), 547–579.
- [36] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph Gonzalez, and Ion Stoica. 2021. Contrastive code representation learning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), ACL, 5954–5971. DOI : <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.482>
- [37] Ashish Jaiswal, Ashwin Ramesh Babu, Mohammad Zaki Zadeh, Debapriya Banerjee, and Fillia Makedon. 2020. A survey on contrastive self-supervised learning. arXiv:2011.00362. Retrieved from <https://arxiv.org/abs/2011.00362>
- [38] Xue Jiang, Zhuoran Zheng, Chen Lyu, Liang Li, and Lei Lyu. 2021. TreeBERT: A tree-based pre-trained model for programming language. In *Proceedings of the 37th Conference on Uncertainty in Artificial Intelligence (UAI '21)*. Cassio P. de Campos, Marloes H. Maathuis, and Erik Quaeghebeur (Eds.), Proceedings of Machine Learning Research, Vol. 161, AUAI Press, 54–63. Retrieved from <https://proceedings.mlr.press/v161/jiang21a.html>
- [39] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and evaluating contextual embedding of source code. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)* Proceedings of Machine Learning Research, Vol. 119, PMLR, 5110–5121. Retrieved from <http://proceedings.mlr.press/v119/kanade20a.html>

- [40] Rafael-Michael Karampatsis and Charles Sutton. 2020. SCELMo: Source code embeddings from language models. arXiv:2004.13214. Retrieved from <https://arxiv.org/abs/2004.13214>
- [41] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick S. H. Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '20)* Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.), ACL, 6769–6781. DOI : <https://doi.org/10.18653/V1/2020.EMNLP-MAIN.550>
- [42] Iman Keivanloo, Juergen Rilling, and Ying Zou. 2014. Spotting working code examples. In *Proceedings of the 36th International Conference on Software Engineering, (ICSE '14)* Pankaj Jalote, Lionel C. Briand, and André van der Hoek (Eds.), ACM, New York, NY, 664–675. DOI : <https://doi.org/10.1145/2568225.2568292>
- [43] Otávio Augusto Lazzarini Lemos, Adriano Carvalho de Paula, Felipe Capodifoglio Zanichelli, and Cristina Videira Lopes. 2014. Thesaurus-based automatic query expansion for interface-driven code search. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*. Premkumar T. Devanbu, Sung Kim, and Martin Pinzger (Eds.), ACM, New York, NY, 212–221. DOI : <https://doi.org/10.1145/2597073.2597087>
- [44] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL '20)*. Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault (Eds.), ACL, 7871–7880. DOI : <https://doi.org/10.18653/V1/2020.ACL-MAIN.703>
- [45] Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. CodeRetriever: Unimodal and bimodal contrastive learning. arXiv:2201.10866. Retrieved from <https://arxiv.org/abs/2201.10866>
- [46] Xuan Li, Zerui Wang, Qianxiang Wang, Shoumeng Yan, Tao Xie, and Hong Mei. 2016. Relationship-aware code search for JavaScript frameworks. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '16)*. Thomas Zimmermann, Jane Cleland-Huang, and Zhendong Su (Eds.), ACM, New York, NY, 690–701. DOI : <https://doi.org/10.1145/2950290.2950341>
- [47] Xiujun Li, Xi Yin, Chunyuan Li, Pengchuan Zhang, Xiaowei Hu, Lei Zhang, Lijuan Wang, Houdong Hu, Li Dong, Furu Wei, et al. 2020. Oscar: Object-semantics aligned pre-training for vision-language tasks. In *Proceedings of the 16th European Conference on Computer Vision (ECCV '20)*. Springer, 121–137.
- [48] Fang Liu, Ge Li, Yunfei Zhao, and Zhi Jin. 2020. Multi-task learning based pre-trained language model for code completion. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. IEEE, 473–485. DOI : <https://doi.org/10.1145/3324884.3416591>
- [49] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692. Retrieved from <http://arxiv.org/abs/1907.11692>
- [50] Meili Lu, Xiaobing Sun, Shaowei Wang, David Lo, and Yucong Duan. 2015. Query expansion via WordNet for effective code search. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER '15)*. Yann-Gaël Guéhéneuc, Bram Adams, and Alexander Serebrenik (Eds.), IEEE Computer Society, 545–549. DOI : <https://doi.org/10.1109/SANER.2015.7081874>
- [51] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021*. Joaquin Vanschoren and Sai-Kit Yeung (Eds.), Retrieved from <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html>
- [52] Fei Lv, Hongyu Zhang, Jian-Guang Lou, Shaowei Wang, Dongmei Zhang, and Jianjun Zhao. 2015. CodeHow: Effective code search based on API understanding and extended boolean model (E). In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE '15)*. Myra B. Cohen, Lars Grunske, and Michael Whalen (Eds.), IEEE Computer Society, 260–270. DOI : <https://doi.org/10.1109/ASE.2015.42>
- [53] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. DOI : <https://doi.org/10.1017/CBO9780511809071>
- [54] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader-Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE '21)*. IEEE, 336–347. DOI : <https://doi.org/10.1109/ICSE43902.2021.00041>

- [55] Collin McMillan, Mark Grechanik, Denys Poshyvanyk, Chen Fu, and Qing Xie. 2012. Exemplar: A source code search engine for finding highly relevant applications. *IEEE Transactions on Software Engineering* 38, 5 (2012), 1069–1087. DOI: <https://doi.org/10.1109/TSE.2011.84>
- [56] Collin McMillan, Mark Grechanik, Denys Poshyvanyk, Qing Xie, and Chen Fu. 2011. Portfolio: Finding relevant functions and their usage. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE '11)*. Richard N. Taylor, Harald C. Gall, and Nenad Medvidovic (Eds.), ACM, New York, NY, 111–120. DOI: <https://doi.org/10.1145/1985793.1985809>
- [57] Ishan Misra and Laurens van der Maaten. 2020. Self-supervised learning of pretext-invariant representations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '20)*. Computer Vision Foundation/IEEE, 6706–6716. DOI: <https://doi.org/10.1109/CVPR42600.2020.00674>
- [58] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*. Dale Schuurmans and Michael P. Wellman (Eds.), AAAI Press, 1287–1293. DOI: <https://doi.org/10.1609/AAAI.V30I1.10139>
- [59] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and code embeddings by contrastive pre-training. arXiv:2201.10005. Retrieved from <https://doi.org/10.48550/arXiv.2201.10005>
- [60] Liming Nie, He Jiang, Zhilei Ren, Zeyi Sun, and Xiaochen Li. 2017. Query expansion based on crowd knowledge for code search. arXiv:1703.01443. Retrieved from <http://arxiv.org/abs/1703.01443>
- [61] Changan Niu, Chuanyi Li, Bin Luo, and Vincent Ng. 2022a. Deep learning meets software engineering: A survey on pre-trained models of source code. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI '22)*. Luc De Raedt (Ed.), ijcai.org, 5546–5555. DOI: <https://doi.org/10.24963/IJCAI.2022/775>
- [62] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguang Huang, and Bin Luo. 2022b. SPT-Code: Sequence-to-sequence pre-training for learning source code representations. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 1–13. DOI: <https://doi.org/10.1145/3510003.3510096>
- [63] Luca Ponzanelli, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, and Michele Lanza. 2014. Mining stackoverflow to turn the IDE into a self-confident programming prompter. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*. Premkumar T. Devanbu, Sung Kim, and Martin Pinzger (Eds.), ACM, New York, NY, 102–111. DOI: <https://doi.org/10.1145/2597073.2597077>
- [64] Varot Premtoon, James Koppel, and Armando Solar-Lezama. 2020. Semantic code search via equational reasoning. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '20)*. Alastair F. Donaldson and Emina Torlak (Eds.), ACM, New York, NY, 1066–1082. DOI: <https://doi.org/10.1145/3385412.3386001>
- [65] Filip Radenovic, Abhimanyu Dubey, Abhishek Kadian, Todor Mihaylov, Simon Vandenhende, Yash Patel, Yi Wen, Vignesh Ramanathan, and Dhruv Mahajan. 2023. Filtering, distillation, and hard negatives for vision-language pre-training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 6967–6977.
- [66] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning*. PMLR, 8748–8763.
- [67] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
- [68] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21 (2020), 140:1–140:67. Retrieved from <http://jmlr.org/papers/v21/20-074.html>
- [69] Mohammad Masudur Rahman and Chanchal K. Roy. 2018. Effective reformulation of query for code search using crowdsourced knowledge and extra-large data analytics. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME '18)*. IEEE Computer Society, 473–484. DOI: <https://doi.org/10.1109/ICSME.2018.00057>
- [70] Matthew Richardson and Pedro M. Domingos. 2001. The Intelligent surfer: Probabilistic combination of link and content information in PageRank. In *Proceedings of the 14th International Conference on Neural Information Processing Systems: Natural and Synthetic (NIPS '01)*. Thomas G. Dietterich, Suzanna Becker, and Zoubin Ghahramani (Eds.), MIT Press, 1441–1448. Retrieved from <https://proceedings.neurips.cc/paper/2001/hash/a501bebf79d570651ff601788ea9d16d-Abstract.html>
- [71] Stephen E. Robertson and Karen Spärck Jones. 1976. Relevance weighting of search terms. *Journal of the American Society for Information Science* 27, 3 (1976), 129–146. DOI: <https://doi.org/10.1002/ASL.4630270302>
- [72] Saksham Sachdev, Hongyu Li, Sifei Luan, Seohyun Kim, Koushik Sen, and Satish Chandra. 2018. Retrieval on source code: A neural code search. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI '18)*. Justin Gottschlich and Alvin Cheung (Eds.), ACM, New York, NY, 31–41. DOI: <https://doi.org/10.1145/3211346.3211353>

- [73] Ensheng Shi, Yanlin Wang, Wenchao Gu, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. CoCoSoDa: Effective contrastive learning for code search. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 2198–2210. DOI : <https://doi.org/10.1109/ICSE48619.2023.00185>
- [74] Jeffrey Svajlenko, Judith F. Islam, Iman Keivanloo, Chanchal Kumar Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution*. IEEE Computer Society, 476–480. DOI : <https://doi.org/10.1109/ICSME.2014.77>
- [75] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.), ACM, New York, NY, 1433–1443. DOI : <https://doi.org/10.1145/3368089.3417058>
- [76] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola. 2020. What makes for good views for contrastive learning? In *Proceedings of the Annual Conference on Neural Information Processing Systems 2020 (NeurIPS '20)*. Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.), Retrieved from <https://proceedings.neurips.cc/paper/2020/hash/4c2e5eaae9152079b9e95845750bb9ab-Abstract.html>
- [77] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS '17)*. Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.), 5998–6008. Retrieved from <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [78] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. 2020. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In *Proceedings of the 27th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '20)*. Kostas Kontogiannis, Foutse Khomh, Alexander Chatzigeorgiou, Marios-Eleftherios Fokaefs, and Minghui Zhou (Eds.), IEEE, 261–271. DOI : <https://doi.org/10.1109/SANER48275.2020.9054857>
- [79] Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021b. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. arXiv:2108.04556. Retrieved from <https://doi.org/10.48550/arXiv.2108.04556>
- [80] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. arXiv:2305.07922. Retrieved from <https://arxiv.org/abs/2305.07922>
- [81] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, and Tie-Yan Liu. 2013. A theoretical analysis of NDCG type ranking measures. In *Proceedings of the 26th Annual Conference on Learning Theory (COLT '13)*. Shai Shalev-Shwartz and Ingo Steinwart (Eds.), JMLR Workshop and Conference Proceedings, Vol. 30, JMLR.org, 25–54. Retrieved from <http://proceedings.mlr.press/v30/Wang13.html>
- [82] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021a. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), ACL, 8696–8708. DOI : <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.685>
- [83] Mark Weiser. 1984. Program slicing. *IEEE Transactions on Software Engineering* 4 (1984), 352–357.
- [84] Jun Xia, Lirong Wu, Ge Wang, Jintao Chen, and Stan Z Li. 2021. Proglcl: Rethinking hard negative mining in graph contrastive learning. arXiv:2110.02027. Retrieved from <https://doi.org/10.48550/arXiv.2110.02027>
- [85] Huang Xie, Okko Räsänen, and Tuomas Virtanen. 2023. On negative sampling for contrastive audio-text retrieval. arXiv:2211.04070. Retrieved from <https://arxiv.org/abs/2211.04070>
- [86] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval. arXiv:2007.00808. Retrieved from <https://doi.org/10.48550/arXiv.2007.00808>
- [87] Shuhan Yan, Hang Yu, Yuting Chen, Beijun Shen, and Lingxiao Jiang. 2020. Are the code snippets what we are searching for? A benchmark and an empirical study on code search with natural-language queries. In *Proceedings of the 27th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '20)*. Kostas Kontogiannis, Foutse Khomh, Alexander Chatzigeorgiou, Marios-Eleftherios Fokaefs, and Minghui Zhou (Eds.), IEEE, 344–354. DOI : <https://doi.org/10.1109/SANER48275.2020.9054840>
- [88] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Lizhen Cui, and Quoc Viet Hung Nguyen. 2022. Are graph augmentations necessary? Simple graph contrastive learning for recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1294–1303.
- [89] Jintao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. Optimizing dense retrieval model training with hard negatives. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1503–1512.
- [90] Jintao Zhan, Jiaxin Mao, Yiqun Liu, Min Zhang, and Shaoping Ma. 2020. RepBERT: Contextualized text embeddings for first-stage retrieval. arXiv:2006.15498. Retrieved from <https://arxiv.org/abs/2006.15498>

- [91] Zhilu Zhang and Mert R. Sabuncu. 2018. Generalized cross entropy loss for training deep neural networks with noisy labels. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems 2018 (NeurIPS '18)*. Montréal, Canada, Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett (Eds.), 8792–8802. Retrieved from <https://proceedings.neurips.cc/paper/2018/hash/f2925f97bc13ad2852a7a551802feca0-Abstract.html>
- [92] Jing Zhou and Robert J. Walker. 2016. API deprecation: A retrospective analysis and detection method for code examples on the web. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '16)*. Thomas Zimmermann, Jane Cleland-Huang, and Zhendong Su (Eds.), ACM, New York, NY, 266–277. DOI: <https://doi.org/10.1145/2950290.2950298>
- [93] Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. 2022. XLCoS: A benchmark dataset for cross-lingual code intelligence. arxiv:2206.08474. Retrieved from <https://arxiv.org/abs/2206.08474>
- [94] Qihao Zhu, Zeyu Sun, Xiran Liang, Yingfei Xiong, and Lu Zhang. 2020. OCoR: An overlapping-aware code retriever. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. IEEE, 883–894. <https://doi.org/10.1145/3324884.3416530>

Received 1 January 2024; revised 10 August 2024; accepted 12 August 2024